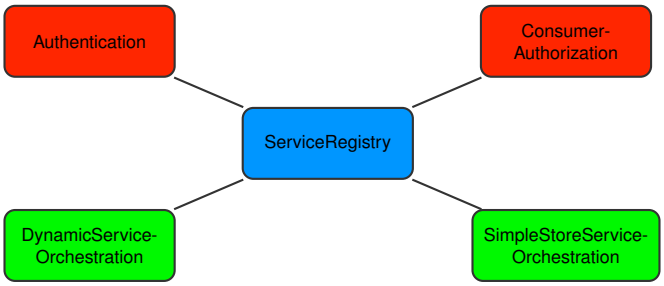
Generic System-of-Systems Description (GSoSD)

The Arrowhead Core and Support Systems 5.2.2



Abstract

This document outlines the 5th generation of the Arrowhead Core and Support systems as realised at release 5.2.2. The Core systems offer functionality expected to be needed for the majority of use cases where Arrowhead is applied: discovery, authentication, authorisation and orchestration. The Support systems offer functionality that many, but not all, deployments need: orchestration support, engineering and automation, credential life cycle, events, data, time, configuration, monitoring and inter-cloud connectivity. The document contains a high-level architectural description of what problems these systems solve and how they interact, including the management layer formed by the PlantDescriptionEngine and its engineering user interface, and the security mechanisms of the framework, to which the Onboarding system adds controlled admission of devices and systems.

The primary purpose of the document is to present the functionality offered by the Arrowhead Core and Support systems. Its architectural aspects are defined in the abstract: no specific implementation or technology is endorsed. Where the 5.2.2 implementations made the abstract design concrete – service names, operations, policies – this document follows them, and refers to the white-box GSoSDDs [1, 2] and to the per-system SysD, SD and IDD documents for implementation detail. The document supersedes *GSoSD Arrowhead Core Systems 5.0 v1.2* [3] and the draft *GSoSD Arrowhead Support Systems 5.0* [4].

Contents

1 Overview **4**
1.1 Implementation baseline and roadmap status (30 October 2026) . . . 4
1.2 Graphical configuration and operational boundaries . . . 5

2 Microsystems **8**
2.1 System Descriptions . . . 8
2.2 Local Cloud Bootstrapping . . . 11
2.3 The System-Service Matrix . . . 12

3 Support Microsystems **13**
3.1 Overview and Release Status . . . 13
3.2 Common Properties of Support Systems . . . 13
3.3 Orchestration Support . . . 15
3.4 Engineering and Automation . . . 15
3.5 Security and Life Cycle . . . 16
3.6 Information and Messaging . . . 16
3.7 Monitoring and Analytics . . . 17
3.8 Inter-Cloud . . . 17
3.9 Deploying Support Systems in a Local Cloud . . . 18
3.10 Deployed Java Reference Cloud (24 October 2026) . . . 19
3.11 Principal Deployed Service Paths . . . 20

4 Management Layer and Security Mechanisms **21**
4.1 The Management Layer . . . 21
4.2 Security Mechanisms of the Arrowhead Framework . . . 22

5 Use-cases **23**
5.1 Service Discovery . . . 23
5.2 Authenticated Access . . . 23
5.3 Authorisation . . . 24
5.4 Orchestration . . . 25
5.5 Authorisation and Orchestration . . . 26
5.6 Onboarding and Off-boarding . . . 26
5.7 Plant-Description-Driven Orchestration . . . 27
5.8 Workflow Execution . . . 28
5.9 Orchestration with Translation . . . 29
5.10 Event-Driven Communication . . . 29
5.11 Monitoring of the Local Cloud . . . 30
5.12 Inter-Cloud Service Consumption . . . 30

6 Control and Data Planes **32**

7 Interoperability with the Java Reference Implementation **32**
7.1 Evidence and Current Deployment Scope . . . 33
7.2 Contract and Security Requirements . . . 34
7.3 Boundaries and Acceptance of Further Replacements . . . 34

8 Release Notes **34**
8.1 From Arrowhead 4.x to 5.0 . . . 34
8.2 From 5.0 (GSoSD v1.2) to 5.2.2 . . . 35
8.3 Support Systems from 4.6 to 5.2 . . . 35

9 Non-Functional Requirements **36**
9.1 Accurate Enough Message Caching . . . 36
9.2 Relative System Independence . . . 36
9.3 Backwards Compatibility . . . 37
9.4 Authentication Mechanisms . . . 37
9.5 Naming Convention for Local Clouds, Microsystems, Microservices, . . . 37
9.6 Active vs. Passive Systems . . . 37
9.7 Port Numbering . . . 38

10 Unresolved Issues **39**

11 References **40**

12 **40**

13 Revision History **42**
13.1 Amendments . . . 42
13.2 Quality Assurance . . . 43

A System-Service Matrix **44**
A.1 Core Systems . . . 44
A.2 Support Systems . . . 44
A.3 Implementations . . . 45

1 Overview

Graphics colour convention. All diagrams use the SysML v1.6 Papyrus library's system-role fills, taken from `EclipseArrowhead-5.0/Arrowhead-5.0.notation`: registry blue, authentication and authorisation red-orange, orchestration green, support yellow and application orange. Both orchestration implementations share the library's combined-cloud green; infrastructure is grey and mixed-system groups are neutral. These colours identify roles rather than implementation languages. The library's turquoise is reserved for service-interface elements.



1.1 Implementation baseline and roadmap status (30 October 2026)

The architecture is realised by three primary source distributions: `Eclipse-Arrowhead-java`, `Eclipse-Arrowhead-go` and `Eclipse-Arrowhead-rust`, under the Eclipse workspace. These are the sources for subsequent implementation changes; older `Arrowhead/` and `EA-fork/` checkouts are historical references or deployment consumers. A primary-source designation does not imply that every running image has been rebuilt from that tree. Release v5.2.2, local extensions, independently packaged sources and dated deployment evidence remain distinct [5].

Language	Current source/build boundary	Evidence and limits
Java	Five core systems in `ah5-core-java-spring`; fifteen independent support projects with own JAR/image/configuration. The Java-cloud manager is Node.js/nginx, despite its folder name.	Support packaging verified on 25 October; core/support services and operations tools have separate lifecycles. No shared JavaSupportRuntime dependency in the fifteen standalone projects.
Go	Five independent core modules and eighteen current top-level support/test/tool modules; private helper copies and optional `go.work`.	28 October audit: 22 module builds, tests and vet checks passed. Packaging does not establish complete Java parity or a new live rollout.
Rust	Five core entry points; fifteen support entry points and a separate LocalCloudManager crate in one Cargo workspace. Core/support libraries and messaging remain shared build dependencies.	28 October copy report: 147 workspace tests passed. Per-system folders are not independently copied source closures. Java-equivalence and operational qualification remain open.

Table 1: Current implementation structure, not a language-equivalence claim. Dated evidence is retained in each primary distribution [6, 7, 8].

Core inventory versus default cloud. Five core implementations remain available. A *new Java local-cloud design* selects ServiceRegistry, Authentication, ConsumerAuthorization and DynamicServiceOrchestration. SimpleStoreServiceOrchestration (labelled SimpleOrchestration by the configurator) is optional and absent from that default; it has not been deleted or implicitly stopped in an existing cloud. This is a configuration default, not a removal of the simple-store architecture or a new upstream release requirement [9].

Database ownership and access boundary. Every defined database is an *integrated asset of its owning microsystem*, including a separately deployed SQL or graph-database container. It is not selectable shared infrastructure or an independent service provider. Only host internal actions or services produced/consumed by that host mediate data access. The ownership rule also applies to embedded SQLite and private snapshot files. Brokers, relays and the constrained Docker proxy remain infrastructure. The Java catalog implements this rule for all sixteen defined database services; the default canvas therefore contains four microsystems, each with its own database, and one AMQP broker. The database assets do not become extra graph nodes [10].

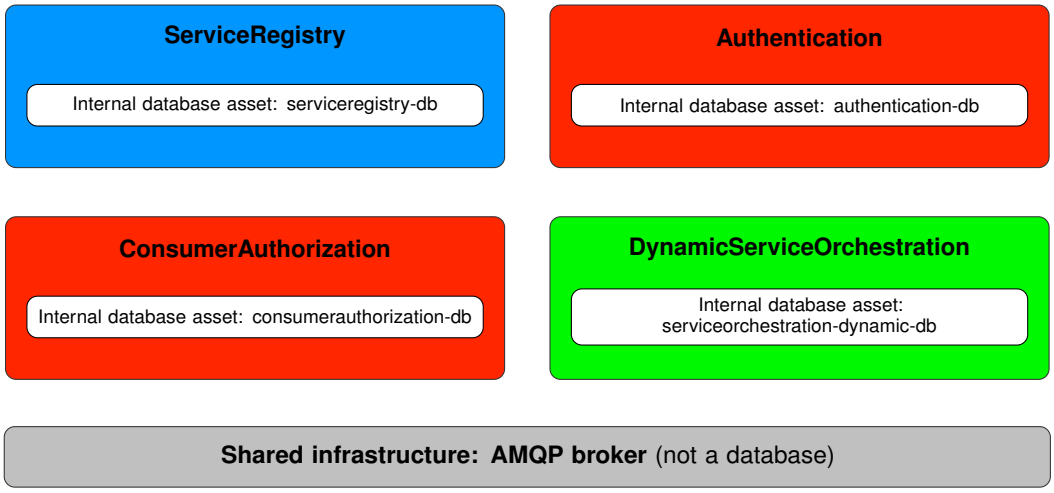


Figure 1: New Java default design. Each database is nested inside its owning microsystem boundary; it is not shared infrastructure or an independently selectable component. Colors retain the established Arrowhead system roles.

Evidence-qualified roadmap. Recorded 19–24 October interoperability and federation results below remain historical acceptance of the stated workloads. The 23 October operation-count audit has not been replaced by a new complete differential audit. The Go packaging audit records a failing optional native-registry integration test: orchestration service registration is rejected, including with unmodified reference sources. Rust still has partial DTO/error/side-effect coverage, unsupported integrations and no automatic Java-state import. Complete replacement requires contract, policy, persistence and mixed-cloud acceptance for the exact image/configuration, beyond successful packaging tests [7, 8, 11].

1.2 Graphical configuration and operational boundaries

The Java LocalCloudManager now offers three related engineering functions. Local-cloud composition selects microsystems and necessary infrastructure, stores named graphical designs, reviews the resulting actions and applies a confirmed deployment workflow. Its SoS view edits systems, service ports, connections and included SoS descriptions through PlantDescriptionEngine; activation is explicit and the PDE remains authoritative. The orchestration view edits local routing rules and ConsumerAuthorization policies graphically. Display positions are presentation data, not authorization grants.

LocalCloudConfigurator configures one cloud; SoSConfigurator instead plans catalogued inter-cloud exchanges and their neighbour/policy changes. Both are operations tools using ColonyOS executors. They are not additional Arrowhead core systems and do not automatically register a discovery identity. An operator must review the plan before applying it. Saving a design does not change running systems; absence from a design does not implicitly stop an existing system [9, 12].

The 30 October Java deployment inspection found 44 running containers in `arrowhead-v52-test-cloud`: 23 microsystem processes, sixteen integrated database containers, two brokers, the manager UI, the Docker proxy and the LocalCloudConfigurator API. The microsystem count includes the optional GraphDBManager extension and Python demonstration producer; the manager API is counted once. GraphDBManager has no implementation in the supplied primary Java, Go or Rust source inventory; its existing Java-cloud image is a deployment extension, not evidence of a new portable support implementation. The older 22-system/41-container figure below is explicitly the 24 October snapshot [13].

The Eclipse Arrowhead project provides a SOA/microservice architecture infrastructure enabling the creation of an implementation platform for a certain application domain. This platform facilitates the implementation of end-user solutions, as depicted in Figure 2.

The focus of Eclipse Arrowhead is microsystems that produce and consume microservices at the edge. The Eclipse Arrowhead architecture and reference implementations target deployment on heterogeneous devices

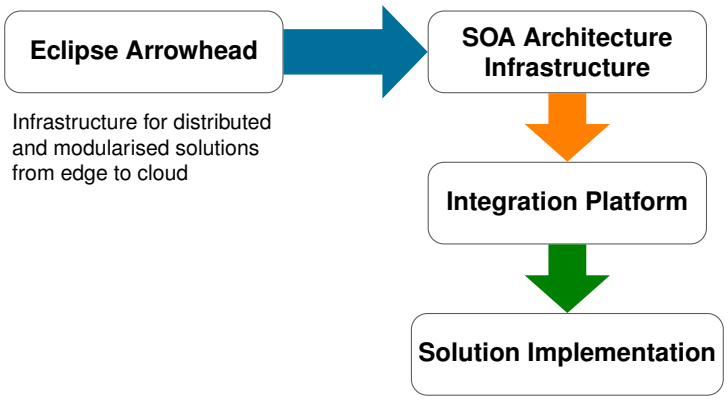


Figure 2: Eclipse Arrowhead as SOA architecture infrastructure for integration platforms and solution implementations.

from deep edge to fog, with interoperability to cloud microservices.

The Eclipse Arrowhead project exists to provide solutions and specifications that support industrial and societal automation solutions that can adapt quickly and automatically to changing circumstances and business requirements. This adaptability is enabled through Service-Oriented Architecture (SOA), which entails dividing automation systems into smaller independent systems that communicate by sending messages to, or consuming, the services offered by the other systems. Each system is expected to be designed around the principles of

- **loose coupling**, which means that services have narrow, well-defined application areas and depend internally on as few other services as possible to fulfil their functions,
- **late binding**, which entails providing as much functionality as possible given what needed services, configuration data and other details are currently available, as well as
- **lookup**, which means that systems, when possible, look up needed data by consuming services instead of relying on the data being in given configuration files.

These principles help facilitate the design of automation systems that are cheap and easy to implement, upgrade, extend and reuse, are highly resilient to failures and attacks, and are highly transparent about their constituents and activities.

While designing industrial automation systems around these principles has the advantages mentioned, it also introduces the complexity inherent to all distributed applications. The Core microsystems of Eclipse Arrowhead exist to help manage this complexity by facilitating

- **discovery**, or helping microsystems independently find the services they need,
- **authentication**, or establishing the identity of the microsystems involved,
- **authorisation**, or regulating how microsystems are allowed to consume what microservices, as well as
- **orchestration**, or centrally managing what microsystems consume what microservices.

Each of these concerns is addressed by its own microservices, provided by its own microsystem, as outlined in the component diagram of Figure 3. Further concerns that are common but not universal – such as engineering-driven orchestration, workflows, credential life cycle, events, data, time, configuration, monitoring and inter-cloud connectivity – are addressed by the Support microsystems.

A deployment of microsystems interrelated in this manner makes up a System-of-Systems (SoS). When a System-of-Systems can fulfil its core functionality without external support, we refer to it as a *local cloud*. Local clouds enable the segmentation of complex SoS. Such segmentation results in multiple local clouds which can interact with each other; such a system of multiple local clouds is referred to as a *system of local clouds*.

Microservices shall be referenced by their microservice type, i.e. their *service definition*. Since release 5.2 the names of microsystems, microservices, operations and interface templates follow the agreed naming convention [14] (Section 9.5).

In Section 2 we consider what the Core services and systems do in more detail, and in Section 3 the Support systems. Section 4 describes the management layer and the security mechanisms of the framework. In Section 5 we describe use cases in which some or all of them are used. In Section 6 we discuss how Core and

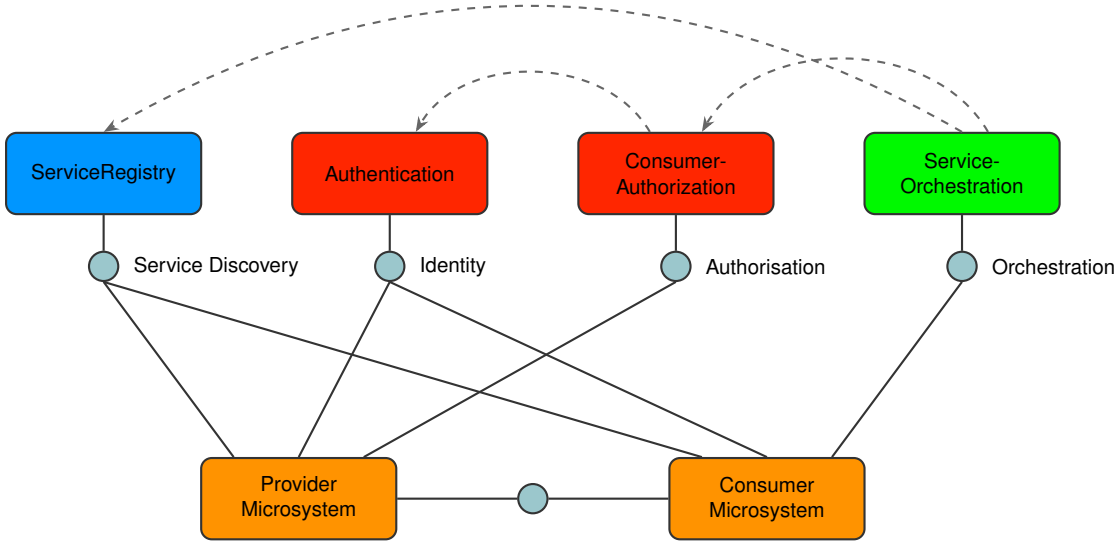


Figure 3: Core microsystems of a local cloud. Each box is a microsystem; a circle is a provided microservice; solid lines show which application systems consume which Core services, dashed arrows consumption between Core systems (registration of every system in the ServiceRegistry is omitted). ServiceOrchestration stands for either of the two orchestration microsystems of Section 2.1.4.

Support systems relate to the control and data planes of a distributed application. In Section 8 we outline the differences between Arrowhead 4.x, 5.0 and 5.2.2. In Section 9 we present key non-functional requirements, and in Section 10 the issues that are currently open. Finally, Sections 11 and 13 list references and versions. Appendix A contains the system-service matrix.

2 Microsystems

The Core systems of the Eclipse Arrowhead project address concerns expected to be typical of most settings where SOA is applied. We describe them here in the abstract. Release 5.2.2 provides five Core microsystems; their mandatory and optional status in a local cloud and their microservices are listed in Table 2. Full AA and encrypted communication are the default from v5.0; optional orchestration or Support features do not make authentication or authorisation optional.

Microsystem	Status	Microservice	Purpose
ServiceRegistry	mandatory	*serviceDiscovery*	Register, look up and revoke service instances
		systemDiscovery	Register, look up and revoke systems
		deviceDiscovery	Register, look up and revoke devices
		serviceRegistryManagement	Bulk administration of all registry content
Authentication	mandatory	*identity*	Log in, log out, change credentials, verify identities
		identityManagement	Administration of identities and sessions
ConsumerAuthorization	mandatory	*authorization*	Grant, revoke, look up and verify access rules
		authorizationToken	Generate and verify access tokens
		authorizationManagement	Administration of access rules
		authorizationTokenManagement	Administration of tokens and encryption keys
DynamicService-Orchestration	optional, one of the two	*serviceOrchestration*	Orchestration pull and push (subscribe, unsubscribe)
		serviceOrchestrationPush Management	Administration of push subscriptions
		serviceOrchestrationLock Management	Administration of exclusivity locks
		serviceOrchestrationHistory Management	Orchestration history
SimpleStoreService-Orchestration	optional, one of the two	*serviceOrchestration*	Orchestration pull and push (same interface)
		serviceOrchestrationSimpleStore Management	Administration of orchestration rules
		serviceOrchestrationPush Management	Administration of push subscriptions
		serviceOrchestrationHistory Management	Orchestration history
All Core systems	–	*generalManagement*	Logs and configuration of the system

Table 2: Core microsystems and microservices of release 5.2.2 under the default v5.0-and-later AA security baseline. Orchestration strategy remains a deployment choice.

The purposes of the respective Core systems are presented in Section 2.1. In Section 2.2 we consider how local clouds reach a working state from a cold start, and in Section 2.3 we introduce the system-service matrix of Core and Support systems.

In addition to the Core systems, several *Support* systems are provided. We expect them to be common in many Arrowhead deployments, but since they are not as essential as the Core systems, they are described separately in Section 3. Three of them are invoked by the advanced Orchestration system.

2.1 System Descriptions

We now state the purposes and services of the Core systems informally. More detailed descriptions of the services are given in the Service Descriptions (SDs), their realisations in the Interface Design Descriptions (IDDs), and the internal design of the 5.2.2 systems in the white-box GSoSDD [1].

2.1.1 ServiceRegistry

One of the main principles of SOA is lookup: whenever possible, data required for whatever reason should be looked up by consuming a service. One kind of data that is virtually always required to facilitate a system-of-systems is the network information, such as addresses and port numbers, needed by systems to communicate with each other.

The **ServiceRegistry** holds the registry of the local cloud. It maps the identifiers of individual service instances to the information generally required to communicate with their providers. Since 5.x the registry also records the *systems* that provide services and the *devices* that host systems, so that the physical deployment of the local cloud can be expressed and queried. A service instance is identified by its provider system, service definition and version; each instance has one or more *interfaces*, each an instance of an *interface template* (for example HTTPS or MQTTS in the default secure profile) that defines which interface properties must be given. Each interface also states the *security policy* a consumer must satisfy, such as certificate-based access or a particular kind of access token.

Individual systems register, update or revoke their own devices, systems and services through the *deviceDiscovery*, *systemDiscovery* and *serviceDiscovery* services, and other systems look them up through the same services. Alternatively, the registry can be populated by an operator through the *serviceRegistryManagement* service, which allows registry content to be created, updated, removed and queried in bulk and is meant primarily for administration.

When using a **ServiceRegistry**, the only network address systems are required to know is that of the **ServiceRegistry**. With that, systems can look up the network addresses of the systems they need to consume, given that they know their service definitions or instance identifiers. Entries are not limited to network details: they may include metadata relevant for selecting service instances, such as the physical location of temperature sensors or the conveyor belt at which a welding service is provided. Lookups can filter on metadata.

A local cloud may *restrict* direct discovery. In restricted mode, ordinary systems can only discover the public Core services directly (those marked for unrestricted discovery, such as *identity*, *authorization* and *serviceOrchestration*); all other services must be obtained through orchestration, which applies authorisation. Systems that need to search the whole registry, such as the orchestration system, are granted direct access by configuration.

2.1.2 Authentication System

In order to enable authorisation to fulfil its purpose in a secure manner, the systems that consume services must be properly identified. The **Authentication** system provides the identity service used by the default AA architecture. Certificate-based peer authentication and identity tokens are complementary mechanisms in the secure reference profile. Identities can be set at deployment time, for example as an X.509 certificate following the Arrowhead certificate profile, or issued dynamically, for example in the form of a token. The **Authentication** system issues time-limited identity tokens after a successful login (*identity*) and validates them on behalf of other systems, so that systems are known to be the ones they claim to be. Identities and their credentials are administered through *identityManagement*.

The implementation supports the following identity mechanisms; only authenticated mechanisms are part of the default cloud profile:

- **certificate** – mutual TLS with Arrowhead-compliant X.509 certificates; the system name is taken from the certificate;
- **identity token** – the system logs in to **Authentication** and presents the token, which providers verify with **Authentication**;
- **self-declared** – the system states its name; intended for development only.

The secure reference profile combines certificates and identity tokens: the token's system name must match the authenticated certificate. A certificate-only service binding must still enforce the applicable authorisation policy and encrypted transport. A limitation of Arrowhead 5.2.2 is that only *system* authentication is performed; user authentication is still under investigation (Section 10).

2.1.3 Authorisation System

Authorisation is part of the default AA security baseline from v5.0. Every service provider must establish which authenticated systems may consume its services and operations, and under which limitations. The Authorisation

system, realised by **ConsumerAuthorization**, holds access control rules (*policies*) that state, for a provider and a service definition (or event type), and optionally per operation, which consumers are permitted. A policy may permit all consumers, a list of consumers (whitelist), all but a list (blacklist), or the consumers whose system metadata satisfies a requirement. Whenever a system attempts to consume a service, the provider enforces a positive authorisation decision, obtained through **ConsumerAuthorization** or a valid service-bound authorisation token, before executing the operation.

Using an Authorisation system means that individual service providers do not need to know in advance what other systems are allowed to consume them. Rules exist on two levels: a provider may grant rules for its own services (provider level, *authorization*), and the local cloud operator manages rules for any service (management level, *authorizationManagement*). **Management-level authorisation rules have priority over the producing application system's own rules**; if no rule applies, access is denied.

ConsumerAuthorization can also issue *access tokens* (*authorizationToken*): time-limited, usage-limited and self-contained (signed, e.g. JSON Web Tokens) tokens bound to a consumer, provider, service and operation. Self-contained tokens can be verified by the provider without contacting **ConsumerAuthorization**. Tokens and provider encryption keys are administered through *authorizationTokenManagement*. To reduce the number of times a system has to consult the Authorisation system, caching may be employed, as discussed in Section 9.1.

2.1.4 Orchestration System

Using only the **ServiceRegistry** means that network information does not have to be manually supplied to the individual systems of a local cloud. However, each system must still be configured, or use its own search procedure, to determine what exact service instances to consume. This becomes an issue when the same type of service is provided by many systems that differ in other respects: if a service instance reports the air quality for a specific location, which instance is consumed affects what data the consumer receives.

The main task of the Orchestration system is to help consumers decide which service instances they should use. An orchestration request can result in a list of providers offering different services (to satisfy all the consumer's needs), a list of providers offering the same service (viable alternatives), or a single provider. In contexts where tokens are used for access control, the Orchestration system also delivers these tokens together with the service instances to consume.

There are two approaches to how an Orchestration system can fulfil this task, with different advantages and disadvantages. Release 5.2.2 provides both, with a shared interface, and the operator of the local cloud decides which kind of orchestration the local cloud uses:

Atomic orchestration – SimpleStoreServiceOrchestration. Holds a table of prioritised rules that map each consumer and service definition to the specific service instances it should consume. An orchestration request returns service instance identifiers in priority order; the consumer uses them to acquire access information from the **ServiceRegistry**. This approach is very simple, uses few computing resources and does not perform the dynamic eligibility pipeline. Resolving a stored rule still requires secure discovery and provider-side authorisation; a rule is not an access grant. Its limitations are that most tasks are delegated to the consumer, that it issues no access tokens, and that keeping the rules consistent with the registry is a task for the operator or a support system (for example the **PlantDescriptionEngine**).

Advanced orchestration – DynamicServiceOrchestration. Computes the result at request time from the consumer's requirements (service definition, operations, interfaces, security policies, metadata, preferred providers, QoS requirements, exclusivity) and flags (matchmaking, only preferred, only exclusive, allow translation, allow or only inter-cloud). It uses ServiceRegistry and the default AA foundation; additional Support systems enable optional capabilities. Its functions include:

- **Authorisation**: providers the consumer is not authorised to use are removed from the response, and the necessary tokens are added automatically;
- **Exclusivity**: providers can be reserved (locked) for one consumer for a period of time;
- **Blacklist**: blacklisted providers are removed;
- **Translation**: if no compatible provider exists, a translation bridge between interfaces or data models is set up through the **TranslationManager**;
- **QoS**: providers are filtered or ranked by the quality of their devices (**DeviceQoSEvaluator**);
- **Inter-cloud**: orchestration across local clouds through **Gatekeeper** and **GateTunnel**.

It is more convenient for consumers, since the response contains everything needed to consume the service, but it is more complex, uses more computing resources and depends on other systems.

Consumers obtain their orchestration results by *pull* (`pull` of *serviceOrchestration*) or subscribe to *push* orchestration (`subscribe`, `unsubscribe`), in which case results are delivered to them when triggered. The orchestration history, push subscriptions, locks and (for the atomic approach) the rule store are administered through dedicated management services.

To address the late binding and loose coupling properties of SOA, orchestrating who can/should/shall talk to whom is important. The orchestration system thus provides rules on who can/should/shall talk to whom; these rules may be based on policies. Such rules can be "owned" by a consumer, a plant description microsystem, a workflow/choreography microsystem, the orchestration microsystem, or an operator (engineering). The hierarchy developed in Arrowhead is

1. Operator – engineering,
2. PlantDescription,
3. Orchestration,
4. Consumer.

For interoperability reasons both push and pull distribution of orchestration rules are supported. Each local cloud has its own orchestration system. In principle there can be multiple orchestration systems in a local cloud, but that creates synchronisation requirements; the recommendation is therefore to have one orchestration system in a local cloud (see Section 10 for a current exception).

Orchestration between local clouds is made possible by the **Gatekeeper** and **GateTunnel** microsystems, in cooperation with the **ServiceRegistry** and the authentication and authorisation microsystems of the other local clouds.

It has to be noted that an application implementation where consumers are "free" to use their "own orchestration" information gives less understanding and control of what goes on in a local cloud, with possible consequences for real-time, security and safety properties. Unforeseen issues may thus emerge.

2.1.5 General Management

Every Core system also provides *generalManagement*, which gives operators access to the system's log and public configuration. Access to all management services of a system is restricted by a configurable management policy: operator only, operator and listed systems, or according to the rules of the Authorisation system.

2.2 Local Cloud Bootstrapping

Since v5.0, full authentication and authorisation (AA) and encryption of exchanged information are the default for an Arrowhead local cloud. Bootstrapping establishes that protected operating state before ordinary service consumption begins. It does not start an unsecured registry-only cloud and add security later. Discovery, registration, identity exchange, policy administration, orchestration and application traffic all remain subject to the default security baseline.

Before startup, every system has a unique Arrowhead identity, the credentials needed to prove it, and the trust material needed to authenticate its peers. Initial credentials and access policies are provisioned by the operator, or obtained through an approved onboarding procedure. The ServiceRegistry bootstrap address must be known or obtained through a trusted mechanism; learning an address does not establish trust in the system at that address.

1. **Establish trust and protected communication.** Provision the local-cloud trust anchors, system/operator credentials and the information needed to check their validity and revocation. Enable encrypted, integrity-protected transport with authenticated peers before exposing any Core endpoint. The reference deployment uses mutual TLS for HTTPS, AMQPS and MQTTS. Credentials and tokens are never exchanged over a plaintext bootstrap channel.
2. **Start the ServiceRegistry as the discovery anchor.** It registers its own services and advertises protected interfaces. Systems may discover the Core endpoints needed to complete authentication and authorisation, but such bootstrap discovery is narrowly scoped and does not waive peer authentication, encryption or management protection.

3. **Make AA services ready.** Start Authentication and ConsumerAuthorization with their bootstrap identities, operator rights and initial policies. In the certificate-bound token profile, systems obtain an identity token over mutual TLS and providers verify its binding to the caller. Authorisation policies govern the provider, service and requested operation; missing permission or an unavailable required verification fails closed.
4. **Enable authorised orchestration.** Start the selected orchestration implementation after its discovery and AA dependencies are ready. SimpleStore uses provisioned consumer-to-service rules; dynamic orchestration evaluates each requirement against eligible, authorised providers. Selection does not remove the provider's responsibility to enforce access control.
5. **Admit Support and application systems.** Each system authenticates, registers its protected service interfaces and consumes services only within its granted rights. Onboarding establishes credentials, not service permissions. Monitoring, management and any inter-cloud path follow the same AA and confidentiality requirements. Support prerequisites are listed in Section 3.9.

These are readiness conditions, not permission to create an anonymous startup window or a rigid process order that ignores implementation dependencies. Bootstrap credentials and scoped policies must break dependency cycles without bypassing AA. Certificate authentication and certificate-bound identity tokens are mechanisms for satisfying the baseline; self-declared names and unencrypted HTTP/MQTT do not satisfy it. Any deliberately unsecured test setup is an explicit, isolated development exception, not the default architecture. A registry-only example can illustrate lookup but is not a complete local-cloud deployment under the v5.0-and-later security baseline.

2.3 The System-Service Matrix

To keep track of what services are produced and consumed by different systems, we use a system-service matrix. It outlines the Core and Support systems in terms of the services they provide and consume, and refers both to the abstract definitions of the services (SDs) and to the available implementations of the systems, whose interfacing details are documented in IDDs. The matrix is given in Appendix A.

3 Support Microsystems

The Core systems address concerns typical of every local cloud. *Support* systems address concerns that many, but not all, deployments have: richer orchestration, engineering-driven configuration of the local cloud, workflow execution, the life cycle of credentials, events, data, time, configuration, monitoring and connectivity between local clouds. Support systems are optional; each is deployed when its concern is present. They behave like any other microsystem of the local cloud: they register in the **ServiceRegistry**, are identified by certificate or identity token, are subject to the rules of the Authorisation system, and are found through discovery or orchestration.

This section replaces the largely unwritten *GSoSD Arrowhead Support Systems 5.0* [4]. It describes the Support systems in the abstract, as for the Core systems in Section 2; the white-box design of each 5.2 Support system is given in [2], and per-system SysD, SD and IDD documents are published with the implementations.

3.1 Overview and Release Status

Two groups of Support systems exist at the 5.2 level:

- **Released with Eclipse Arrowhead 5.2.2** [15]: **Blacklist**, **TranslationManager** and **DeviceQoSEvaluator**. They extend the advanced Orchestration system.
- **Project implementations on the 5.2.2 baseline** [16]: twelve active support systems migrated from Arrowhead 4.6 or newly designed, deployed and verified in the 5.2.2 test beds but not yet reviewed for an upstream release (Section 10). The former delegated SysMonitorPublisher is retained only as historical documentation, not a sixteenth active support implementation.

Figure 4 shows how the Support systems relate to the Core. Three of them are consumed by the advanced Orchestration system; the others consume Core services like any application system, and several of them in turn drive the Core (orchestration rules, push orchestration, credentials).

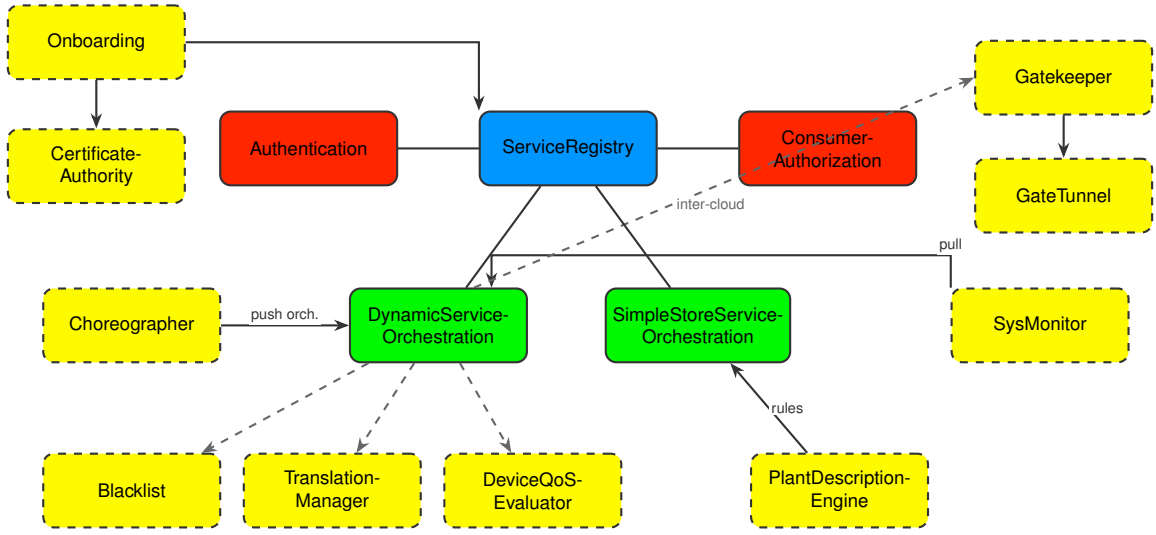


Figure 4: Support systems around the Core (selection). Dashed arrows: consumed by advanced orchestration when the feature is enabled; solid arrows: Support systems driving Core systems or each other. All Support systems also register in the ServiceRegistry and use the authentication and authorisation services; EventHandler, DataManager, TimeManager, Configuration and AIGym are consumed by application systems and omitted.

3.2 Common Properties of Support Systems

All 5.2 Support systems share the following properties, which follow from the Core architecture:

- **Registration and discovery.** Each system registers itself and its services in the **ServiceRegistry** and revokes them on shutdown. Systems that must search the whole registry (**TranslationManager**, **Device**

Area	Microsystem	Microservices	Purpose
Orchestration support	**Blacklist**^{U}	*blacklistDiscovery*, *blacklistManagement*	Exclude misbehaving systems from the local cloud
	TranslationManager^U	*translationBridge*, *translationBridge Management*, *translationReport*	Bridge incompatible interfaces and data models
	DeviceQoSEvaluator^U	*qualityEvaluation*, *deviceQualityData Management*	Rank providers by the quality of their devices
Engineering & automation	**PlantDescriptionEngine**	*plantDescriptionManagement*, *plant DescriptionMonitor*	Derive orchestration rules from an engineered plant description; report deviations
	Choreographer	*choreographerSession*, *choreographer Executor*, *choreographerManagement*	Execute workflows on executor systems
	Configuration	*configuration*, *configurationProxy*, *configurationManagement*	Distribute configuration files to systems
Security & life cycle	**CertificateAuthority**	*certificateSigning*, *certificateValidation*, *certificateAuthorityManagement*	Issue, validate and revoke certificates
	Onboarding	*onboarding*, *onboardingManagement*	Admit (onboard) and withdraw (off-board) devices and their systems
Information & messaging	**EventHandler**	*eventPublish*, *eventSubscription*, *event HandlerManagement*	Publish/subscribe of events
	DataManager	*dataProxy*, *dataHistorian*	Cache and store sensor and actuator data
	TimeManager	*time*	Trusted time for the local cloud
Monitoring & analytics	**SysMonitor**	*sysMonitorHistory*, *sysMonitorData*, *sys MonitorManagement*	Collect and store the operational observations of all microsystems
	AIGym	*aiGym*	Experiment with models on monitoring data
Inter-cloud	**Gatekeeper**	*globalServiceDiscovery*, *interCloud Negotiation*, *gatekeeperManagement*	Find services in, and negotiate access to, other local clouds
	GateTunnel	*gateTunnelConnection*, *gateTunnelKey*, *gateTunnelManagement*	Establish secured sessions between local clouds

Table 3: Support microsystems and microservices at the 5.2 level. ^{U} = released with Eclipse Arrowhead 5.2.2; all others are project implementations. Every Support system also provides *generalManagement* and, as a project proposal, *sysMonitor*.

QoSEvaluator, **Choreographer**, **PlantDescriptionEngine**, **SysMonitor**) are granted direct discovery access.

- **Identity.** Each system is identified by its Arrowhead certificate, optionally combined with an identity token from the **Authentication** system.
- **Authorisation of produced services.** Application services are protected by certificate-based interface policies and, per call, by **ConsumerAuthorization** *authorization*; ownership rules apply where the caller's identity is decisive (a system's own configuration, a session's owner, a step's executor, a bridge's translator).
- **Management.** Management services are protected by the management policy of the system, as for the Core systems, and every system provides *generalManagement*.
- **Own data.** Each system owns its data store; no store is shared.
- **Observability.** Each system publishes periodic liveness, power and status observations through the *sysMonitor* contract (Section 3.7); its liveness reflects its own health, for example "degraded" while its store is unavailable or its time is untrusted.

3.3 Orchestration Support

3.3.1 Blacklist

The **Blacklist** system lets operators and authorised systems exclude named systems from the local cloud for a period of time, with a mandatory reason. A system is excluded while at least one unexpired entry exists for it. Every system that enables blacklist filtering checks each requester with *blacklistDiscovery* and rejects excluded ones; the advanced Orchestration system removes excluded providers from its results. A blacklisted system can still look up why it is excluded. Entries are created, removed and queried through *blacklistManagement*; a configured whitelist of systems, such as the **Blacklist** itself, can never be excluded. Filtering can be configured to fail closed when the **Blacklist** system is unavailable.

3.3.2 TranslationManager

When no provider offers a service in a form the consumer can use – a different protocol or a different data model – the advanced Orchestration system can ask the **TranslationManager** to create a *translation bridge*. The **TranslationManager** discovers *interface translators* that can reach the candidate providers and, if the data models differ, *data model translators*; it negotiates a bridge, obtains the tokens the translator needs, and returns the bridge interface, which the consumer then uses like a normal service interface. Bridges have a life cycle (discovered, initialised, used, closed, aborted, error); translators report usage and termination through *translationReport*. Management of bridges is offered through *translationBridgeManagement*.

3.3.3 DeviceQoSEvaluator

The **DeviceQoSEvaluator** maintains quality data for the *devices* hosting the systems of the local cloud: network round-trip time and, where a device agent is installed, CPU load, memory use and network load. It maps systems to devices from the registry content. Through *qualityEvaluation* the advanced Orchestration system asks it to *filter* providers whose device quality does not meet a threshold, or to *sort* providers by device quality. Measurements and the device map are administered through *deviceQualityDataManagement*.

3.4 Engineering and Automation

3.4.1 PlantDescriptionEngine

A *plant description* is an engineered model of the local cloud: its systems, the services each system consumes or produces (*ports*), and the *connections* between consumer and producer ports, with priorities. The **PlantDescriptionEngine** keeps one plant description active and continuously reconciles the local cloud with it: it matches plant systems to registered systems (by name and metadata), atomically replaces the corresponding managed snapshot in DynamicServiceOrchestration, retaining empty scopes for unavailable producers and withdrawing scopes no longer described. Deviations – a described system that is not registered or is matched ambiguously, a registered system that is not described, a system that is not monitored or has no recent observations – are raised as *alarms* that operators acknowledge. Descriptions are managed through *plantDescriptionManagement*; descriptions and alarms are read through *plantDescriptionMonitor*. In the hierarchy of orchestration rule owners (Section 2.1.4), the **PlantDescriptionEngine** realises the PlantDescription level. Together with its engineering user interface it forms the management layer of the local cloud (Section 4.1); it is operational in the 5.2.2 test beds.

3.4.2 Choreographer

The **Choreographer** executes pre-defined workflows (*plans*). A plan is a chain of *actions*; an action is a graph of *steps*, where each step states a service requirement and parameters, and independent steps run in parallel. *Executors* are application systems that register with the **Choreographer** and perform steps. For each step the **Choreographer** selects an executor and has the advanced Orchestration system orchestrate the required provider *for the executor* by push orchestration, so that authorisation and tokens apply to the executor; it then starts the executor, which reports completion. A failed executor is excluded and the step retried with another one. Clients start and abort sessions through *choreographerSession*; executors use *choreographerExecutor*; plans, sessions, executors and the work log are managed through *choreographerManagement*.

3.4.3 Configuration

The **Configuration** system stores one configuration file per system and lets each system fetch its own file at run time (*configuration*); a system can only read its own configuration. Authorised systems can read and store configurations on behalf of others (*configurationProxy*), and operators manage all configurations (*configuration Management*). Together with lookup, this lets systems keep only the **ServiceRegistry** address and their credentials as local configuration.

3.5 Security and Life Cycle

3.5.1 CertificateAuthority

The **CertificateAuthority** issues the X.509 certificates by which systems are identified. It signs certificate requests submitted by authorised provisioning systems (the **Onboarding** system or an operator), never issues certificates with signing capability, and records every issued certificate. Through *certificateValidation* any system of the local cloud can check whether a certificate was issued by the local cloud and is valid and not revoked, and whether a trusted public key is valid. Revocation and trusted keys are managed through *certificateAuthority Management*.

3.5.2 Onboarding

As the Arrowhead architecture relies on storing metadata about systems for orchestration, choreography and so on, every system in every local cloud must be referable unambiguously; that reference is its *identity*. *Onboarding* is the process by which a new device and the systems it hosts obtain identities and the information they need to join the local cloud; *off-boarding* is its reverse. The **Onboarding** system implements the complete procedure after Bicaku et al. [17]: an operator approves a device by the fingerprint of its manufacturer-issued bootstrap certificate and names the systems it may host (*onboardingManagement*); the device then obtains, step by step through the *onboarding* service, an onboarding certificate, a device certificate and registry entry, and per approved system a system certificate and an identity, each step requiring the credential of the previous one. Off-boarding withdraws all of it. Every certificate is signed by the **CertificateAuthority**; enrolment is by certificate only. Onboarding is the admission-control mechanism of the framework (Section 4.2); the procedure is shown in Section 5.6.

3.6 Information and Messaging

3.6.1 EventHandler

The **EventHandler** decouples producers and consumers of *events*. Systems subscribe (*eventSubscription*) to an event type, optionally with a metadata filter and a validity window, and give a notification address. Publishers publish events (*eventPublish*); the **EventHandler** delivers each event to all matching subscribers. Publisher and subscriber identities are taken from authentication, never from the message. Subscriptions are administered through *eventHandlerManagement*.

3.6.2 DataManager

The **DataManager** handles measurement and actuation data in the SenML format. Its *dataProxy* service keeps the latest values per system and service, so that consumers can read data without contacting the producer; its *dataHistorian* service stores data over time and answers time-window queries per signal.

3.6.3 TimeManager

The **TimeManager** provides the current time, the time zone of the local cloud and an assessment of whether the time can be *trusted* (*time*). It compares its clock with external time servers and reports the time as trusted only while the deviation is below a threshold; it is untrusted until the first successful comparison.

3.7 Monitoring and Analytics

3.7.1 The sysMonitor Contract

Operating a local cloud requires knowing whether its microsystems are alive and healthy. As a project proposal for 5.2, every microsystem – Core, Support and application – provides a *sysMonitor* service: it periodically publishes *observations*, at least `echo` (alive or degraded), `power` and `status`, each carrying the provider, observation type, source time, sequence number, value, unit and quality. Observations are published over AMQP with mutual TLS to a topic exchange of the local cloud's AMQP broker, under the publishing system's own certificate, as persistent messages confirmed by the broker; the routing key names the cloud, the system, the service instance and the observation type [18]. The broker lets each system publish only under its own routing keys and bind its queues only to the keys it is entitled to. The service is registered like any other, with an AMQP interface template, and access to each system's observations is governed by the Authorisation system. The AMQP broker carries only *sysMonitor*; the local cloud's MQTT broker remains in place for all other MQTT traffic. Liveness probes over HTTP, where still offered, are registered as a separate *echo* service.

3.7.2 SysMonitor

The **SysMonitor** system collects the observations of all monitored microsystems. It finds the *sysMonitor* providers through the advanced Orchestration system, so it follows only the providers it is authorised for, receives their observations from the AMQP broker in a durable queue bound to the selected providers and observation types, validates and de-duplicates them, and stores them for a retention period. It provides the stored history (*sysMonitorHistory*), a normalised stream of observations over MQTT (*sysMonitorData*) and operator management (*sysMonitorManagement*), with which operators select the providers and observation types to receive. The **PlantDescriptionEngine** uses the history to detect systems that are not monitored or not alive. (A former **SysMonitorPublisher** published observations on behalf of systems without their own publisher; it is deprecated now that all 5.2 systems publish themselves.)

3.7.3 AIGym

The **AIGym** is a sandbox for experiments with models on local-cloud data. It subscribes to the normalised monitoring stream (*sysMonitorData*, MQTT) and lets authorised systems add labelled samples, obtain predictions and read recent observations (*aiGym*).

3.8 Inter-Cloud

3.8.1 Gatekeeper

Each local cloud that takes part in a system of local clouds has a **Gatekeeper**, its single point of contact for other local clouds. Through *globalServiceDiscovery* it searches neighbouring local clouds for a service; through *interCloudNegotiation* it negotiates with the **Gatekeeper** of a provider's cloud that a local consumer may use a remote provider. The provider's **Gatekeeper** answers only with providers that its own Authorisation system grants to that consumer of that cloud. Neighbour clouds and relays are administered through *gatekeeper Management*.

3.8.2 GateTunnel

When a consumer and a remote provider cannot reach each other directly, the **GateTunnel**s of the two local clouds carry the service exchange of a negotiated *session*: the consumer's **GateTunnel** offers a local endpoint for that consumer only, forwards its requests to the provider's **GateTunnel**, which calls the provider and returns the response. Sessions are opened by the **Gatekeeper**s (*gateTunnelConnection*); *gateTunnelKey* provides the public key for securing a session, and *gateTunnelManagement* lets operators supervise sessions. It succeeds the 4.x Gateway.

3.8.3 The Inter-Cloud Relay

Gatekeepers and **GateTunnel**s of different local clouds communicate through an *inter-cloud relay* [19]: a message broker of the organisation (MQTT with mutual TLS) that every participating local cloud can reach. The relay carries global service discovery and inter-cloud negotiation messages between **Gatekeeper**s, and

the requests and responses of sessions between **GateTunnel**s. Trust between the clouds is established by an *organisation certificate authority* that cross-certifies each local cloud's CA; each cross-certificate is *name-constrained* to its own cloud, so a cloud CA can vouch towards the relay only for identities of its own cloud. **Gatekeeper**s and **GateTunnel**s use dedicated relay identities for this purpose, not their ordinary cloud certificates; inside a local cloud only the cloud's own CA is trusted. The relay's access rules let each **Gatekeeper** and **GateTunnel** read only its own cloud's inbox for its own role and write into another cloud's inbox only under its own identity, so a receiver takes the sender's identity and cloud from the authenticated channel, never from the message body.

3.9 Deploying Support Systems in a Local Cloud

Support systems are started after the Core systems (Section 2.2, step 5). Beyond the general prerequisites – an identity, registration rights and access rules – some Support systems require explicit Core configuration:

- Direct discovery access in the **ServiceRegistry** for systems that search the whole registry (Section 3.2).
- Management rights: the **PlantDescriptionEngine** manages registry queries and orchestration rules; the **Choreographer** manages push orchestration; the advanced Orchestration system and the **Translation Manager** generate tokens on behalf of consumers.
- Access rules for every consumer of a Support service and for the **SysMonitor** on every *sysMonitor* provider.
- The orchestration approach: PDE, workflows and monitoring now use DSO in the local Java profile. PDE requires the managed-rule extension and schema upgrade. SimpleStore may still run independently for clients that explicitly select the atomic approach.
- For onboarding: the **Onboarding** system needs the rights to have certificates signed and revoked, to create and remove identities, to manage registry entries and to exclude systems; approving devices requires an operator.

Canonical ports of all Support systems are allocated by the port numbering of Section 9.7.

3.10 Deployed Java Reference Cloud (24 October 2026)

The local Java reference deployment is the Docker Compose project `arrowhead-v52-test-cloud`, maintained in `Arrowhead-test-cloud-java`. Figure 5 records its 22 deployed microsystems, including both orchestration variants, the operations manager and the demonstration application [20]. This is a dated deployment example of the architecture, not a requirement that every local cloud deploy every optional support system. In this 24 October snapshot, the five Core systems and fifteen support systems are Java implementations; **LocalCloudManager** has a Node.js API and **RandomDataProducer** is a Python application. Thus "Java cloud" identifies the reference implementation baseline, not the language of every process.

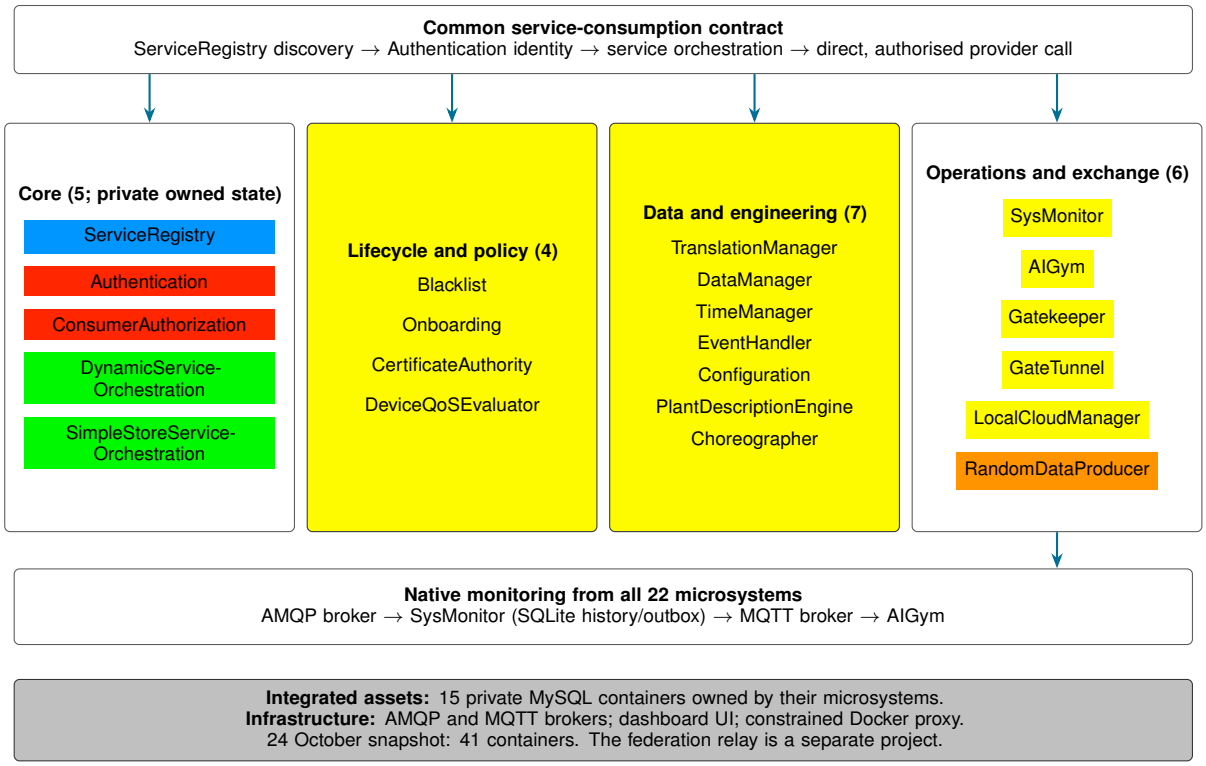


Figure 5: Deployed Java reference cloud. All 22 named systems are microsystems; message brokers, dashboard and Docker proxy are infrastructure; database containers are integrated assets inside their host microsystems. The shared consumption path is drawn once to avoid repeating bootstrap edges.

The 41 running containers comprise the 22 microsystems, 15 dedicated MySQL containers, two message brokers, a dashboard UI and a Docker proxy. The four **CertificateAuthority**, **Gatekeeper**, **GateTunnel** and **Sys Monitor** services originally used the combined Java 21 runtime in this dated snapshot. The primary distribution now packages each as an independent Java 21 project with its own SQLite state; this source change is not a blanket live-image rollout. The former delegated **SysMonitorPublisher** is not a deployed microsystem: each system publishes its own *sysMonitor* observations.

Discovery, orchestration and direct consumption. Systems bootstrap through **ServiceRegistry**, locate *identity* and *serviceOrchestration*, obtain a certificate-bound identity token and request an authorised provider for the required operation. The resulting call goes from consumer to provider; the orchestrator does not proxy the application payload. DSO directly discovers its own authorisation and blacklist dependencies to avoid recursive orchestration. Authentication also bootstraps its authorisation check. Other dependency calls use orchestration, with the documented caches and exceptions.

Monitoring and application data. Native publishers send observations over AMQPS to RabbitMQ; **Sys Monitor** consumes authorised selections and stores them in its own SQLite history/outbox. It republishes

normalised *sysMonitorData* through MQTT for **AIGym**. **LocalCloudManager** reads *sysMonitorHistory* and manages subscriptions. The demonstration producer instead writes application SenML data to **DataManager** every 30 s through *dataHistorian* `put-data`. Monitoring history is not stored by DataManager.

Engineering, workflow and federation paths. The plant editor calls **PlantDescriptionEngine**; an active plant is compiled into an atomic DSO rule snapshot through *serviceOrchestrationPlantManagement*. **Choreographer** asks DSO for push orchestration on behalf of workflow executors; no executor is deployed by default. Onboarding consumes certificate signing, validation and revocation, identity management and blacklist operations. Inter-cloud selection uses Gatekeeper discovery/negotiation and an AH-GT-1 GateTunnel session over the external relay. The 24 October deployment snapshot contained neither SimpleStore route rows nor an active DSO plant-rule row: these mechanisms are available, but the diagram does not imply an active engineered workflow.

Reference security profile. The deployed service profile realises the v5.0-and-later default: full AA and information encryption, using mutual TLS with `CERT_AUTH`, certificate-bound identity tokens and operation-scoped authorisation. Unsecured component samples are explicit development exceptions, not an alternative architectural default. The operator console requires login and reaches Docker only through a project-scoped proxy. CertificateAuthority publishes CRLs; services and brokers enforce them on new TLS handshakes. Ending established connections is a separate action. AH-GT-1 protects relayed payloads end to end; cloud application trust remains separate from relay/organisation trust [20, 21].

3.11 Principal Deployed Service Paths

Figure 6 distinguishes the orchestration decision from the subsequent application call and relates the main optional management and federation services to that path.

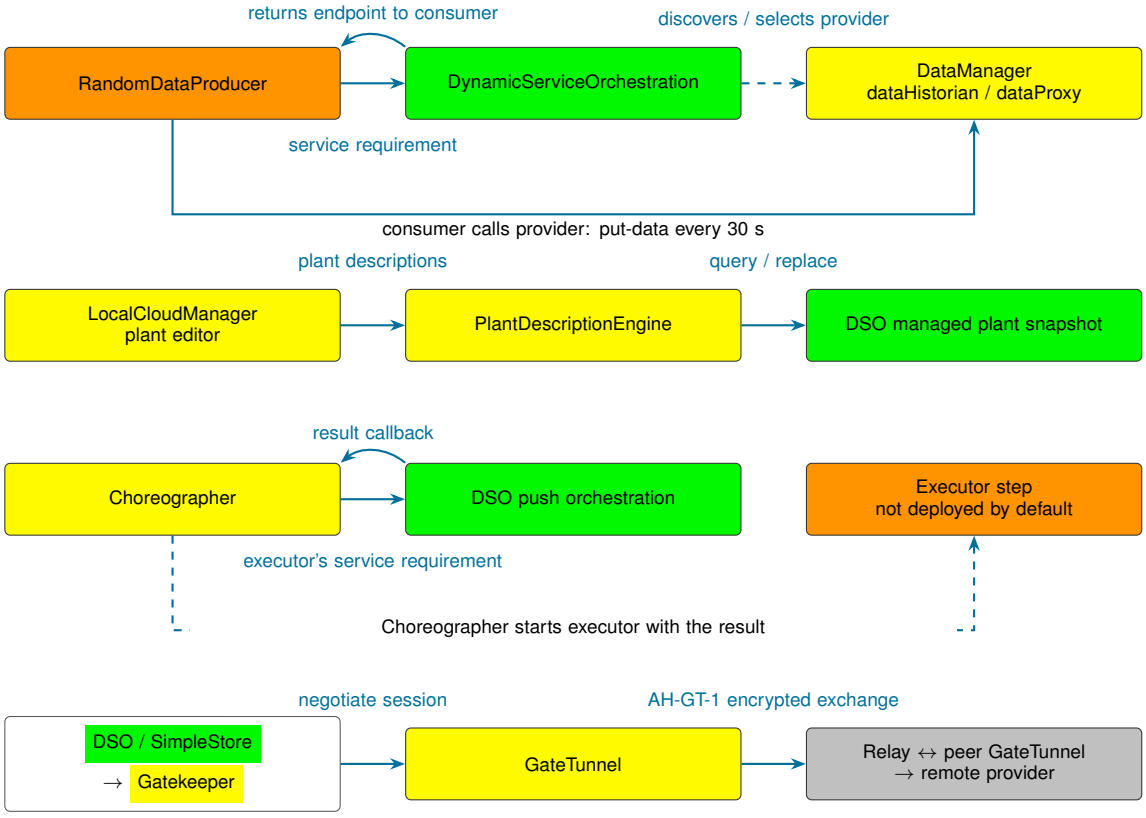


Figure 6: Principal service paths of the deployed Java example. DSO selects and returns a provider; the consumer sends application data directly to that provider. Dashed paths indicate selection or an undeployed executor capability.

4 Management Layer and Security Mechanisms

Two capabilities of the 5.2 framework span several Core and Support systems and are therefore described here as wholes: the *management layer*, through which engineers and operators state how the local cloud shall be composed, and the set of *security mechanisms*, to which the **Onboarding** system adds controlled admission and withdrawal of devices and systems.

4.1 The Management Layer

The Core systems form the control plane of a local cloud (Section 6): they decide, request by request, who may talk to whom and how. What the local cloud *should* look like – which systems belong to the plant and which consumer shall use which producer – is engineering knowledge that lies above the control plane. Release 5.2 makes this a separate, operational *management layer* (Figure 7):

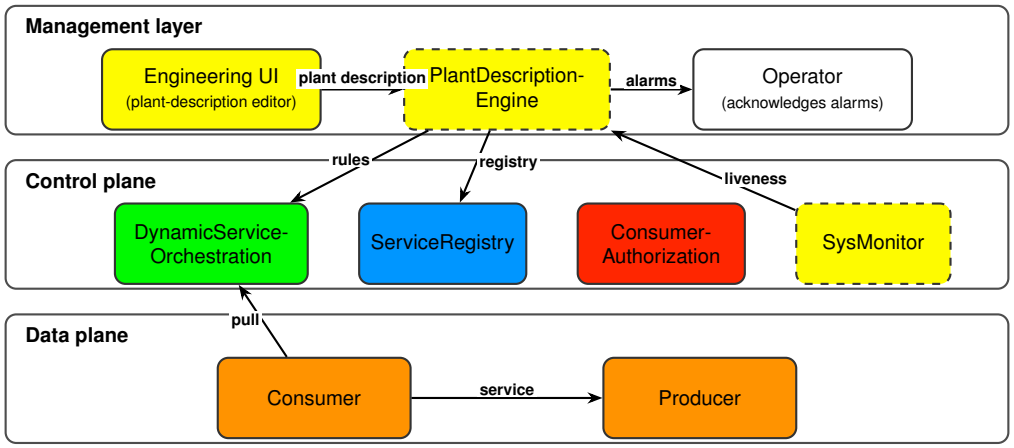


Figure 7: The management layer above the control and data planes. Engineering intent enters as a plant description, becomes orchestration rules, and is checked against the running cloud.

Engineering intent. An engineer describes the plant – its systems, the services each consumes or produces (ports) and the connections between them, with priorities – as a *plant description*. Plant descriptions can include each other, so that a plant can be composed of line or cell descriptions. One description is active at a time.

Enforcement. The **PlantDescriptionEngine** turns the active description into orchestration rules of DynamicServiceOrchestration and keeps them in step with the registry: a connection becomes rules as soon as both its systems are registered and identifiable, and its rules are withdrawn when it is removed from the description. In the hierarchy of rule owners (Section 2.1.4) these rules are the *PlantDescription* level: engineered, therefore above the orchestration system's own decisions and the consumer's preferences, and below explicit operator decisions.

Supervision. The **PlantDescriptionEngine** compares the running local cloud with the description and raises *alarms*: a described system that is missing or ambiguous, a running system that is not described, a system that is not monitored or has gone silent (from the **SysMonitor** history). Operators acknowledge alarms.

Engineering UI. The first engineering user interface of the framework is the plant-description editor of the local-cloud operator console. It lists the stored plant descriptions and edits them in a form – systems and their ports, connections with priorities, includes, activation – or directly as the plant-description document, and stores them through *plantDescriptionManagement*; alarms are read through *plantDescriptionMonitor*, which the first UI does not yet display. A companion view shows, for each provider and service, which consumers are *authorised* and which are *orchestrated* to it, so that engineered intent and access rules can be compared.

The management layer has no privileged path into the local cloud. The engineering UI acts through the **PlantDescriptionEngine**'s management service, and the **PlantDescriptionEngine** acts through the

management services of the **ServiceRegistry** and DynamicServiceOrchestration; each of these is protected by the management policy of the providing system (Section 2.1), and the console is a client that must be granted management rights like any other. The updated Java management layer uses DSO alone; the earlier test-bed setup required SSO next to DSO. Deployment requires the DSO managed-rule extension and schema migration.

4.2 Security Mechanisms of the Arrowhead Framework

Full AA security and encryption of exchanged information are the default from v5.0. They are realised by cooperating mechanisms at different points in a system's life (Table 4); they are not optional additions to discovery. The 5.2.2 description additionally covers explicit admission and withdrawal through **Onboarding**, which complements the existing security baseline.

Mechanism	What it decides	Provided by
Admission and withdrawal (onboarding, off-boarding)	Which devices and systems may obtain credentials at all, and when they lose them	**Onboarding**, **Certificate Authority**
Encrypted communication	Confidentiality and integrity of exchanged information, including bootstrap, management and application traffic	Authenticated TLS endpoints and brokers; end-to-end protection for the selected federation profile
Certificate-based identity	Who is communicating (mutual TLS, local-cloud certificates)	**CertificateAuthority**; every system
Identity tokens	Who is communicating, as a short-lived token bound to the certificate	**Authentication**
Authorisation rules	Who may consume which service and operation; management rules take priority	**ConsumerAuthorization**
Access tokens	A short-lived credential per consumer, provider and service	**ConsumerAuthorization**
Restricted discovery	Which service metadata can be found directly; direct discovery does not waive authentication or provider authorisation	**ServiceRegistry**
Management policy	Who may administer a system	every system
Exclusion	Which systems are barred from the local cloud for a period	**Blacklist**
Inter-cloud trust	Which other local clouds' **Gatekeeper**s and **GateTunnel**s are accepted	Organisation CA (name-constrained cross-certificates), relay access rules, provider-side authorisation

Table 4: Security mechanisms of the Arrowhead Framework 5.2.2.

Onboarding as admission control. Without onboarding, credentials are provisioned out of band, and the local cloud cannot tell which device a system runs on or whether its credential was ever meant to be issued. The **Onboarding** system makes admission an explicit, operator-controlled procedure and builds a chain of trust from the hardware device to its hosted systems:

1. An operator *approves* a device: its manufacturer-issued bootstrap certificate (identified by fingerprint) and the systems it may host. Nothing else can enrol.
2. With the approved bootstrap certificate the device obtains a short-lived *onboarding certificate*; with that, a *device certificate* and its registry entry; with that, per approved system, a *system certificate* and an identity. Each credential can be obtained only with the previous one, and only the latest valid credential of each kind is accepted.
3. The systems then authenticate, register and are authorised like any other system: admission does not grant rights; access rules remain the operator's decision.
4. *Off-boarding*, by the device or the operator, withdraws everything the procedure created: registry entries and identities are removed, certificates revoked, and the systems excluded.

All certificates are signed by the **CertificateAuthority**; the **Onboarding** system holds no signing key. Onboarding thus complements the mechanisms that follow it: authentication and authorisation decide what a credential may do, onboarding decides whether the credential exists. Its withdrawal is fully effective only where the other mechanisms honour it – where revocation is checked and exclusion is enforced (Section 10).

5 Use-cases

Security precondition for every use case. All exchanges below assume the v5.0 default: authenticated participants, positive authorisation and encrypted information exchange. Each diagram isolates one function; omitted TLS handshakes or policy calls are not permission to disable them.

To make it more apparent how the Arrowhead Core and Support systems can be used in practical scenarios, we present twelve use cases: five on the Core systems (Sections 5.1–5.5) and seven involving Support systems (Sections 5.6–5.12). Each centres around a sequence diagram illustrating how systems consume each other's services. Messages are labelled with the service definition and operation that realise them in 5.2.2.

5.1 Service Discovery

The first and most basic use case shows how system B can consume service *a*, provided by A, by looking up its address in ServiceRegistry. The default AA and encryption baseline remains active; this diagram isolates lookup and omits the security exchanges. B has an explicit right to discover this service directly, A and B know the trusted registry's address, and A enforces B's permission before executing *a*. An open discovery setting does not disable authentication or provider access control.

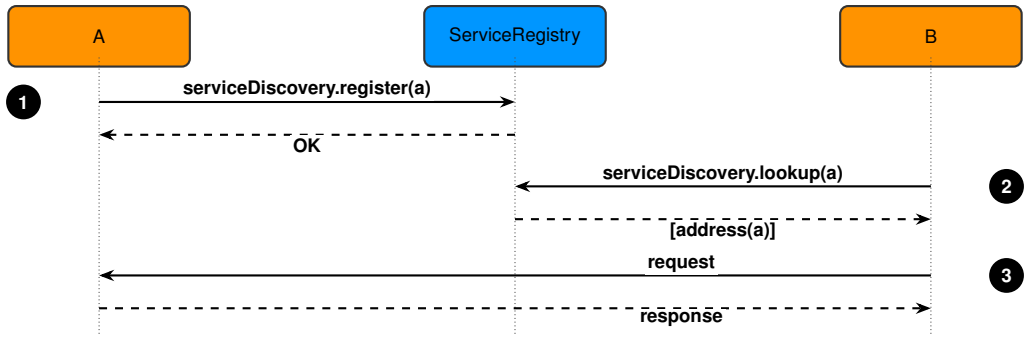


Figure 8: Service discovery.

The use case consists of three message exchanges:

1. A registers its service *a* with the **ServiceRegistry**, which accepts the registration.
2. Later, B looks up *a* via the **ServiceRegistry**, which responds with the network address and other details of *a*.
3. B consumes *a* by sending a request to A, which responds with some status and payload of relevance to the service.

As only one instance of *a* was registered, the response in exchange 2 contained only one address. If more instances had been registered, B would have received all of them and would have had to determine itself which instance is most appropriate, perhaps by looking at metadata in the registrations. Alternatively, B could have looked up a particular service instance identifier instead of a service definition; since identifiers are unique, the result contains at most one entry. This requires the identifier to be known in advance and the instance to be online when B needs it. Another approach is to centralise decisions about service consumption in an Orchestration system (Section 5.4). In a local cloud with restricted discovery, this use case is only available to systems granted direct access.

5.2 Authenticated Access

The second use case shows the identity-token part of AA over an already encrypted, mutually authenticated channel. A and B have trusted local-cloud certificates and pre-registered identities in **Authentication**. TLS setup is omitted from the sequence diagram, not from the security requirements. Authorisation is shown separately in the following use case and remains required before service execution.

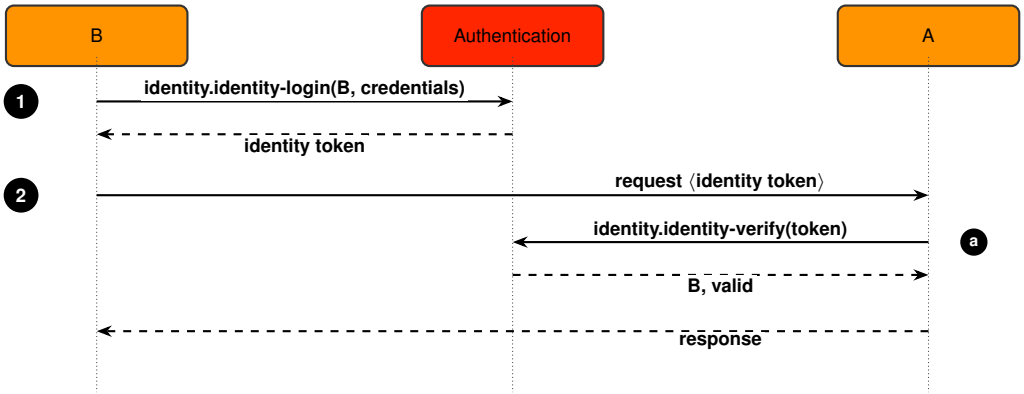


Figure 9: Authenticated access with an identity token.

1. B logs in to the **Authentication** system with its credentials and receives a time-limited identity token. (The address of the *identity* service is obtained from the **ServiceRegistry**, where it is always directly discoverable.)
2. B sends its request to A and includes the identity token.
   (a) A asks the **Authentication** system to verify the token and learns that the requester is B; only then does A process the request.

In the default reference token profile, A also requires the verified token identity to match B's authenticated certificate. A certificate-only binding derives identity from mutual TLS instead of exchanges 1 and 2a; it still requires authorisation and encryption. Neither mechanism makes a self-declared identity sufficient.

5.3 Authorisation

In this use case we add access control through the Authorisation system. Whenever an attempt is made to consume a service, the providing system validates the request by consulting **ConsumerAuthorization** before responding. We assume that all services were registered in the **ServiceRegistry** before the use case begins, that the requesters are authenticated as in Section 5.2, and that **ConsumerAuthorization** has been provided with an appropriate set of access rules.

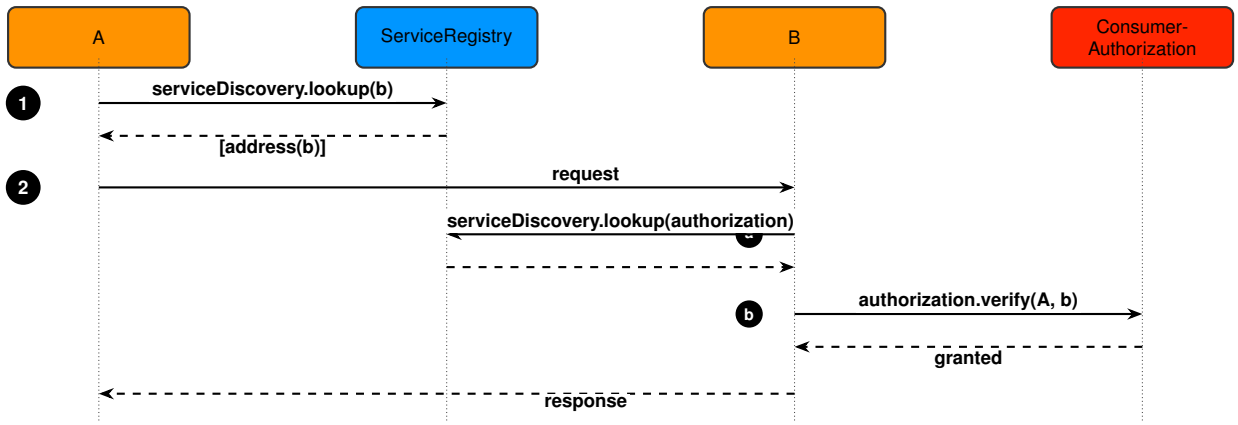


Figure 10: Authorisation by the provider.

1. A looks up *b* via the **ServiceRegistry**, which responds with the network address of *b*. This requires A to have direct discovery access; in a local cloud with restricted discovery A would use orchestration instead (Section 5.5).

2. A attempts to consume *b* by sending a request to B, which later responds. Before responding, B must ascertain that A is permitted to consume *b*:
   (a) Since B does not yet know the network address of **ConsumerAuthorization**, it looks it up; the *authorization* service is always directly discoverable.
   (b) B asks **ConsumerAuthorization** whether A is permitted to consume its instance of *b*, which it is. Management-level rules are evaluated first and take priority over B's own provider-level rules.

Using authorisation in this manner means that access rules must be prepared and provided to **Consumer Authorization** in advance. As rules contain the unique identifiers of the systems and services of the local cloud, those identifiers must be known in advance, and every system must be preconfigured with its own identity. The need for preconfiguring rules can be reduced by rules based on system metadata, or by a separate system that creates access rules dynamically according to given strategies (for example the **PlantDescriptionEngine**).

Compared with the 5.0 description, the **ServiceRegistry** no longer consults the Authorisation system on each lookup; instead its discovery policy restricts what can be discovered directly, and the Authorisation system is consulted for management operations and by orchestration.

5.4 Orchestration

As noted in Section 5.1, using only a **ServiceRegistry** means that we must rely on either (1) systems being able to choose what specific service instances to consume, or (2) preconfiguring each system with the identifiers of those instances. With an Orchestration system, both can be avoided by having the systems ask what services they should consume. We assume that all services were registered before the use case begins and, for atomic orchestration, that the Orchestration system has been provided with appropriate orchestration rules. Authentication, authorisation and encrypted transport remain active; their exchanges are omitted here to isolate service selection.

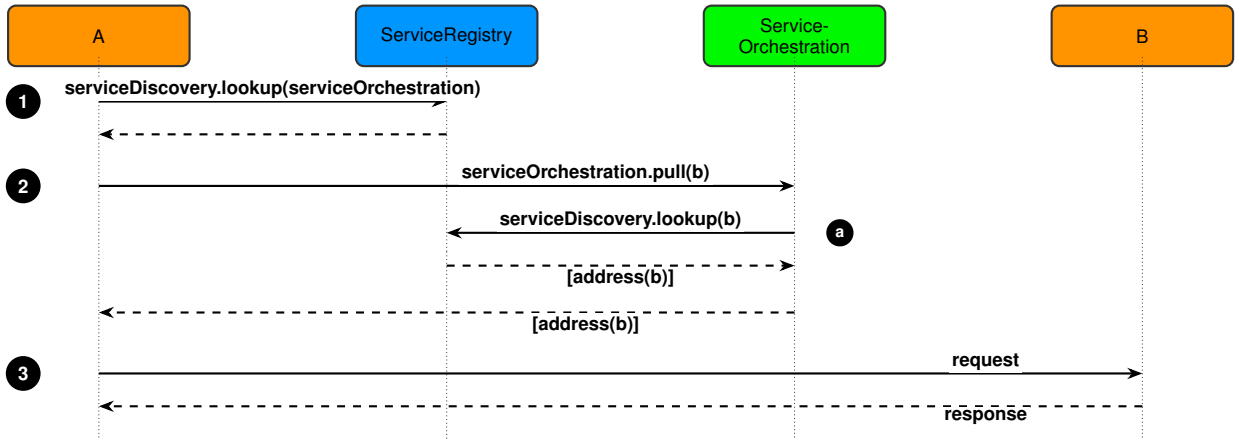


Figure 11: Orchestration (pull), advanced approach. With atomic orchestration, step 2a is omitted, the response contains instance identifiers only, and A resolves them in the **ServiceRegistry**.

1. A looks up the network address of the *serviceOrchestration* service through the *serviceDiscovery* service of the **ServiceRegistry**.
2. A asks the Orchestration system which service instances it should consume, and the Orchestration system responds with the network address of *b*, provided by B.
   (a) Since the advanced Orchestration system holds no network addresses itself, it consults the **Service Registry**, to which it has direct access.
3. A consumes *b* of B by sending a request, which B responds to.

In this use case the orchestration information was *pulled* by A. The Orchestration system also supports *push*: A subscribes to orchestration for *b* and receives results whenever the subscription is triggered, for example when

its rules change. Pushing means that changes reach their systems more reliably and more quickly than periodic pulls.

5.5 Authorisation and Orchestration

There is one important reason to consider authorisation and orchestration together: the advanced Orchestration system can filter providers by the consumer's access rights and distribute access tokens, which increase security because the credentials actually passed between systems expire quickly. We assume that all services were registered before the use case begins, that A has already looked up the address of the Orchestration system, and that **ConsumerAuthorization** has been provided with an appropriate set of access rules.

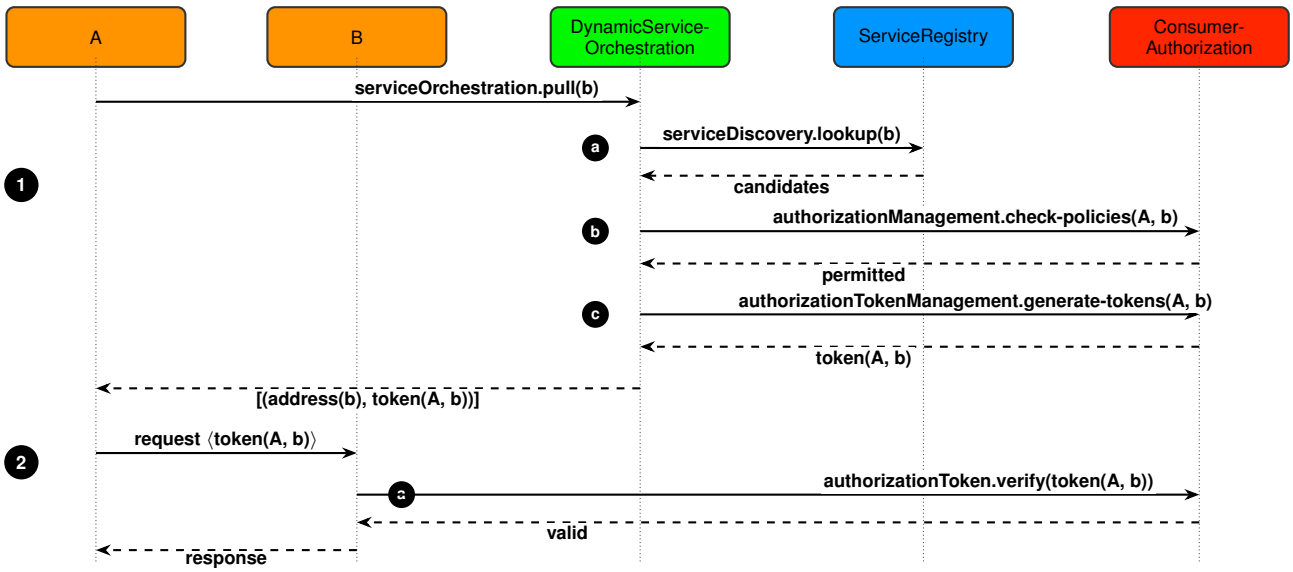


Figure 12: Authorisation and orchestration with access tokens.

1. A requests orchestration for *b* from **DynamicServiceOrchestration**, which responds with the network address of *b* and the access token A needs to consume it.
   (a) The Orchestration system determines the candidate instances of *b* by consulting the **Service Registry**.
   (b) It checks with **ConsumerAuthorization** which candidates A is permitted to consume, and removes the others.
   (c) For the remaining candidates whose interface requires a token, it asks **ConsumerAuthorization** to generate an access token for A. (The orchestration system is permitted to request tokens on behalf of consumers.)
2. A consumes *b* and includes the access token in the request. B responds.
   (a) Before responding, B verifies the token with **ConsumerAuthorization**. A self-contained (signed) token can instead be verified locally with **ConsumerAuthorization**'s public key.

A is identified throughout by its certificate or identity token (Section 5.2). In 5.0 the Orchestration system first checked whether A was authorised to request orchestration; in 5.2.2 *serviceOrchestration* is public to authenticated systems, and authorisation is applied to the providers returned.

5.6 Onboarding and Off-boarding

The onboarding process of the 5.0 Support GSoSD [4] follows Bicaku et al. [17]: a new device proves that it is expected, obtains an identity from the system that maintains identities, and registers its systems and

services. Release 5.2.2 realises the complete procedure, including off-boarding. The announcement steps of the 2018 procedure are replaced by pre-distributed addresses (Section 10), and identities are X.509 certificates of increasing scope: onboarding, device and system certificates.

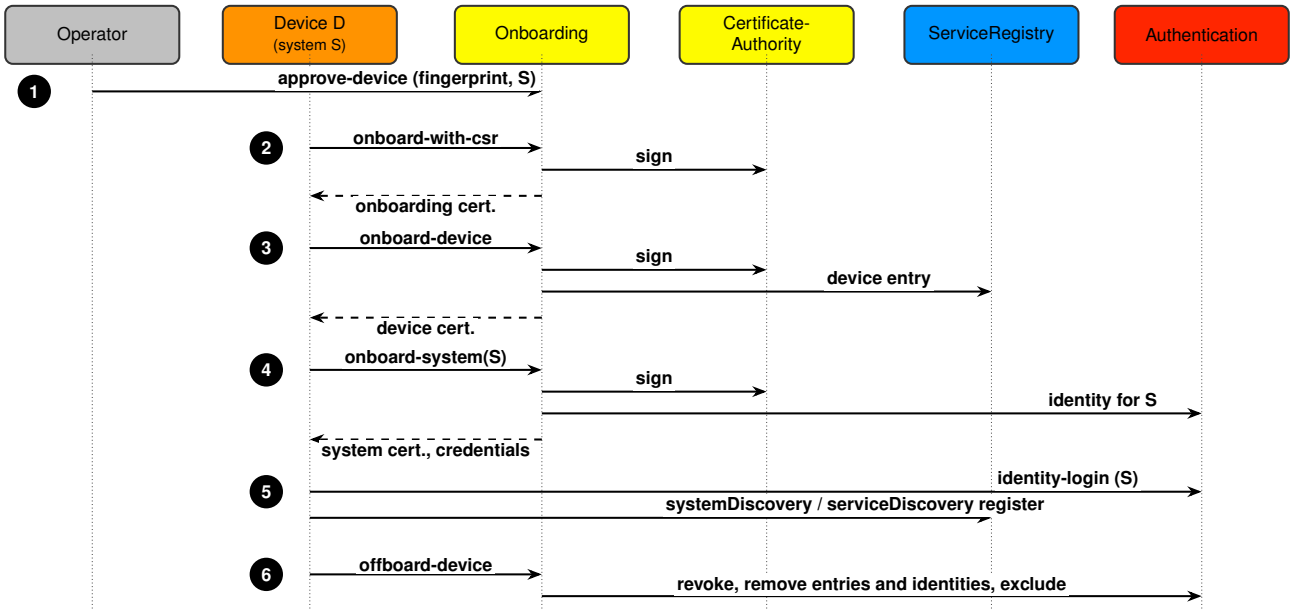


Figure 13: Onboarding and off-boarding of a device D hosting a system S (operations of *onboarding* and *onboarding Management*).

1. An operator approves device D: the fingerprint of its manufacturer-issued bootstrap certificate and the systems it may host, here S. Nothing that is not approved can enrol.
2. D presents its bootstrap certificate and obtains a short-lived onboarding certificate (signed by the **CertificateAuthority**) and the descriptions of the registry services.
3. With the onboarding certificate D obtains its device certificate; the **Onboarding** system registers D as a device in the **ServiceRegistry**.
4. With the device certificate D obtains, for each approved system, a system certificate and an identity in the **Authentication** system.
5. S logs in, registers itself (naming its device D) and its services, and from then on acts as any other system; what S may consume or provide is decided by access rules as usual.
6. Off-boarding, requested by D or by the operator, withdraws everything: certificates are revoked, registry entries and identities removed, and S is excluded through the **Blacklist**.

Each step accepts only the credential issued in the previous step, and only the latest valid one; a superseded or revoked credential is refused.

PDE backend update, 23 October 2026. The Java PDE now submits its complete engineered plant to DynamicServiceOrchestration, which constrains each managed consumer/service to eligible engineered providers and their priorities. An unavailable producer yields no result rather than an unrelated provider. Admission and service-authorisation requirements still apply. SSO remains optional and is no longer required by PDE. The white-box GSoSDDs specify the local extension and coordinated upgrade; existing consumer caches may delay visible changes. This records implementation changes, not proof of a new live deployment.

5.7 Plant-Description-Driven Orchestration

An engineer describes the connections of a plant in the engineering UI (Section 4.1) and lets the **Plant DescriptionEngine** turn them into orchestration rules. We assume all systems are registered and monitored.

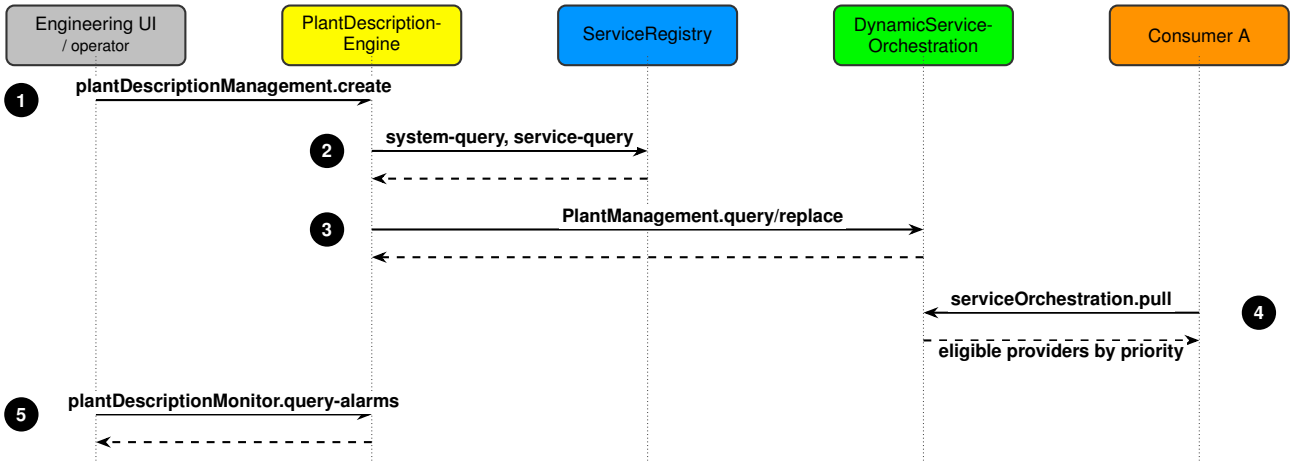


Figure 14: Plant-description-driven orchestration.

1. The engineer creates and activates a plant description in the engineering UI, which stores it with the **PlantDescriptionEngine**.
2. The **PlantDescriptionEngine** matches the described systems with the registered ones.
3. It atomically replaces the managed consumer/service scopes and provider priorities for the active plant; unavailable producers retain empty scopes. Steps 2–3 repeat periodically and after every change.
4. A consumer obtains its orchestration from DynamicServiceOrchestration as in the dynamic-orchestration contract; authorisation and eligibility checks still apply.
5. The operator reads the alarms raised for deviations between the plant and its description, including systems without recent monitoring observations.

The engineer stays in control of the plant's composition: changing the description in the UI changes DSO's managed snapshot within one successful synchronisation interval. Consumer orchestration caches may delay visible changes; no consumer reconfiguration is required.

5.8 Workflow Execution

A client runs a workflow plan. We assume the plan and its executors are registered with the **Choreographer**, and that access rules exist for the executors on the step services.

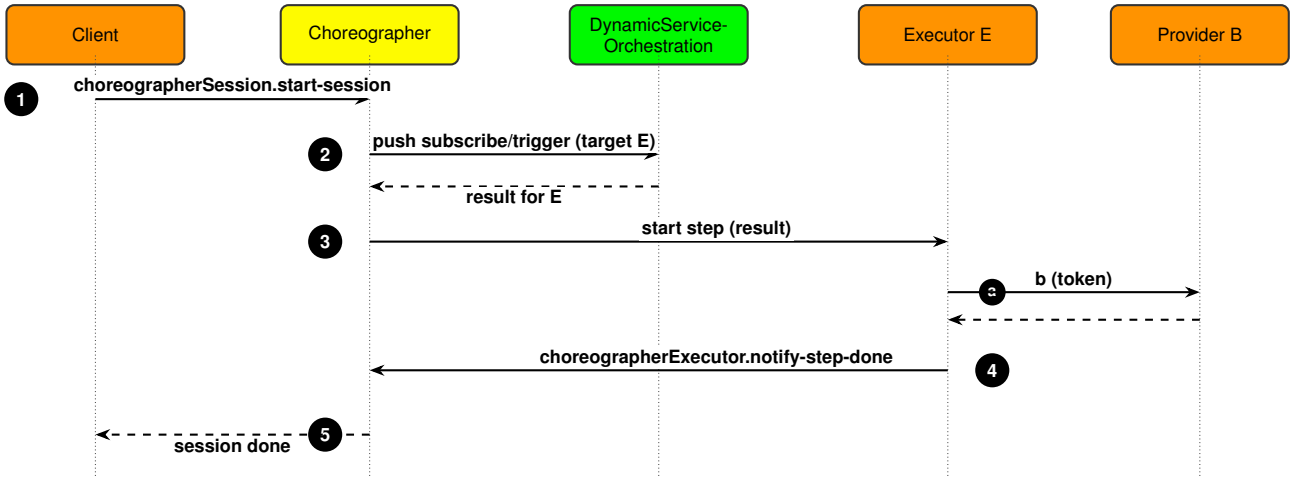


Figure 15: Execution of one workflow step.

1. The client starts a session of a plan.
2. For each step ready to run, the **Choreographer** selects an executor E and has the advanced Orchestration system orchestrate the step's service requirement for E; the result, including tokens issued to E, is pushed back.
3. The **Choreographer** starts E with the orchestration result.
   (a) E consumes the provider's service with its own identity and token.
4. E reports the step result; on error the step is retried with another executor.
5. When all actions and iterations are done, the session ends.

5.9 Orchestration with Translation

Consumer A requires a service that is only provided with an interface or data model A cannot use. Translation is enabled in the advanced Orchestration system and allowed in A's request.

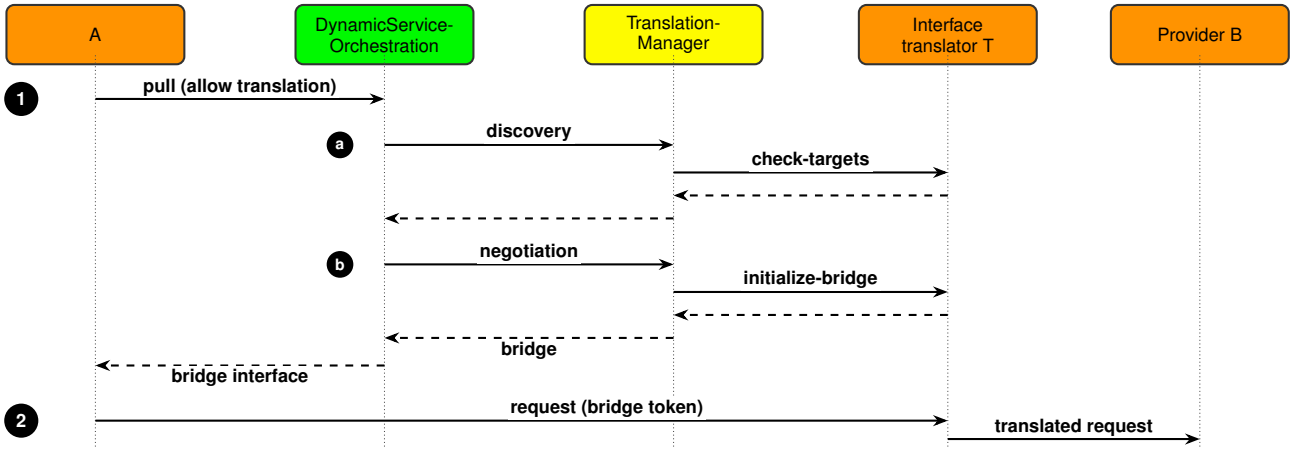


Figure 16: Orchestration with an interface translation bridge (responses omitted in step 2).

1. A requests orchestration and allows translation. No provider is directly compatible.
   (a) The Orchestration system asks the **TranslationManager** to discover possible bridges; it asks interface translators which candidate providers they can reach and, if needed, finds data model translators.
   (b) The Orchestration system selects a candidate and asks the **TranslationManager** to negotiate the bridge, which initialises it in the translator and returns the bridge interface, protected by a bridge token.
2. A consumes the bridge interface like a normal service; the translator forwards translated requests to B and reports usage to the **TranslationManager**.

5.10 Event-Driven Communication

1. S subscribes to an event type with an optional metadata filter, validity window and notification address.
2. P publishes an event; its identity is taken from authentication.
3. The **EventHandler** delivers the event to every subscriber whose filter and validity window match.

Publisher and subscribers never need to know each other. Access control on event types is an open issue (Section 10).

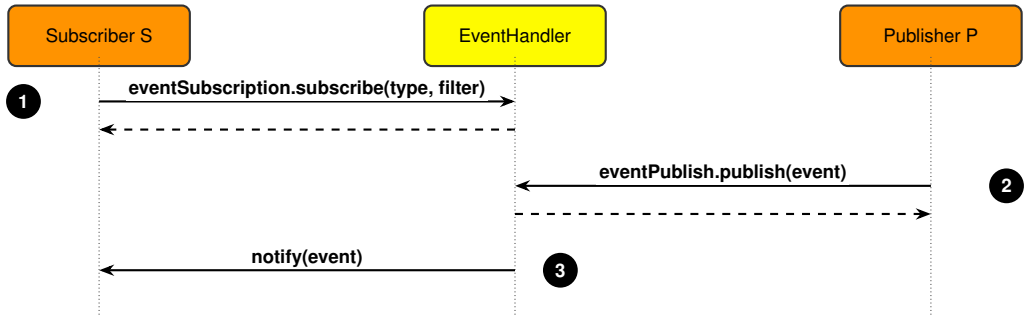


Figure 17: Event-driven communication.

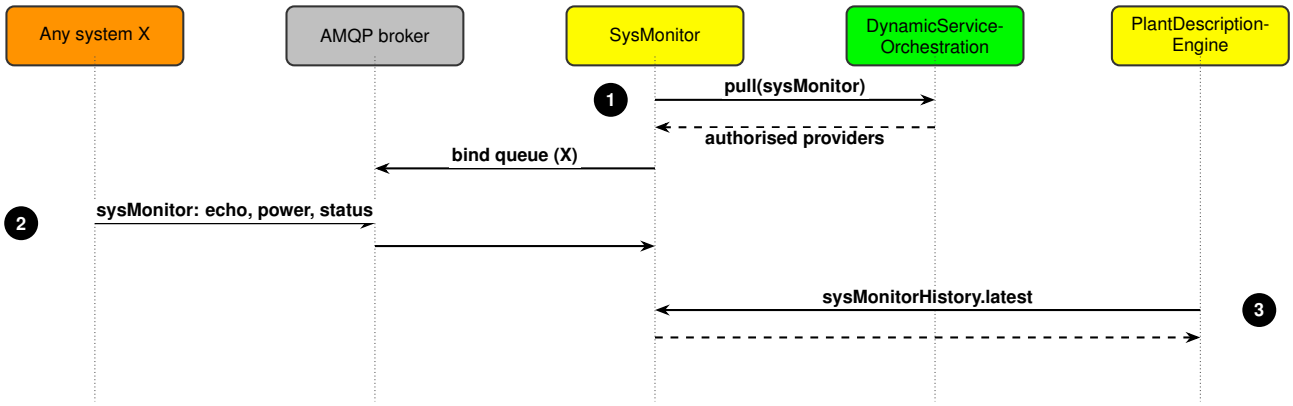


Figure 18: Monitoring of the local cloud.

5.11 Monitoring of the Local Cloud

1. The **SysMonitor** obtains the *sysMonitor* providers it is authorised for through orchestration and binds its queue at the AMQP broker to their observations.
2. Every system X publishes its observations periodically to the AMQP broker; the **SysMonitor** receives, validates and stores them, and republishes them in normalised form over MQTT (*sysMonitorData*).
3. Consumers such as the **PlantDescriptionEngine** read the history to detect systems that are degraded, not monitored or silent.

5.12 Inter-Cloud Service Consumption

Consumer A in local cloud 1 needs a service *b* that only provider B in local cloud 2 offers. Both clouds belong to the same organisation and are connected by an inter-cloud relay (Section 3.8.3); cloud 2 has granted A of cloud 1 access to *b*.

1. A requests orchestration for *b*, allowing or requiring providers in other clouds.
2. The advanced Orchestration system of cloud 1 asks its **Gatekeeper** for a global service discovery. The request reaches the **Gatekeeper** of cloud 2 over the relay; it looks *b* up in its own registry and keeps only the providers its Authorisation system grants to A of cloud 1.
3. The Orchestration system has its **Gatekeeper** negotiate access to B. The provider's **Gatekeeper** checks authorisation again; both **Gatekeeper**s have their **GateTunnel**s open the two ends of a session. The orchestration result given to A is the local session endpoint of **GateTunnel** 1.
4. A consumes *b* at that endpoint with its own credentials, as it would a local provider. **GateTunnel** 1 forwards the request over the relay; **GateTunnel** 2 accepts it only from its negotiated peer and only for B's service, calls B and returns the response.

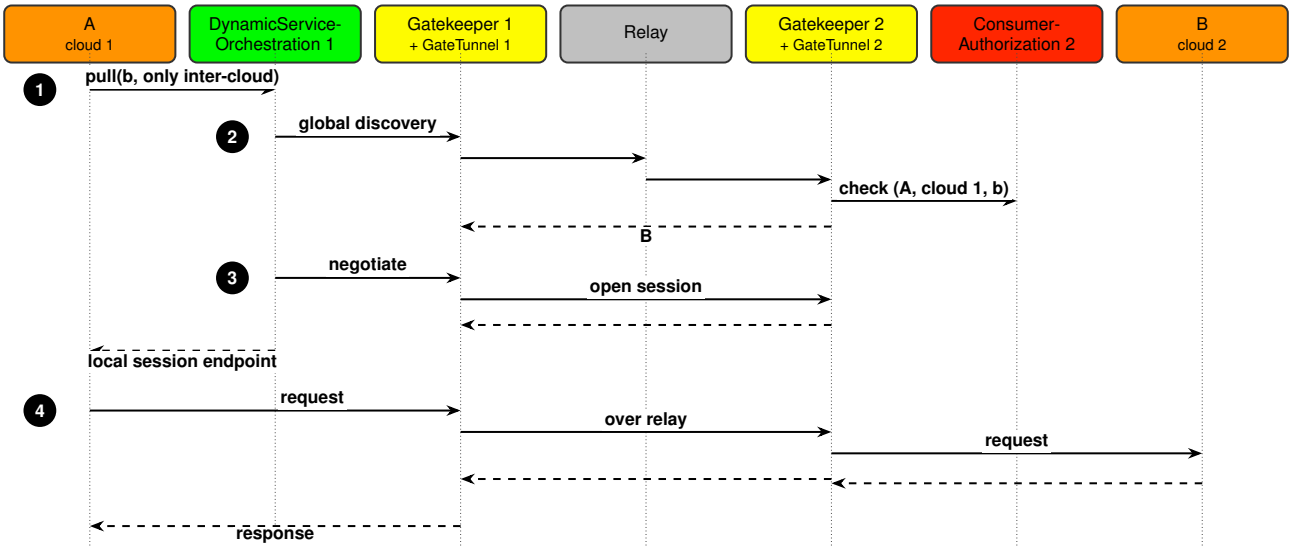


Figure 19: Inter-cloud service consumption through Gatekeepers, GateTunnels and the relay.

No system of cloud 1 needs credentials of cloud 2, and neither cloud exposes its internal network: both clouds connect outwards to the relay.

6 Control and Data Planes

None of the Core and Support systems provided by the Eclipse Arrowhead project are designed to directly fulfil the business objectives of any given local cloud. What they are designed to do is to help create the situation in which these business objectives can be fulfilled. They do this, generally, by controlling and distributing information about the local cloud and its resources.

Traditionally, computer networks are reasoned about in planes, where each plane presents the same network from a different perspective with its own aims and conditions. The term *control plane* is used when considering how networking equipment, such as firewalls, switches and routers, is set up to allow information to flow in certain ways. The term *data plane* is used when considering the contents of those information flows, without regard to how they were set up. This separation increases the number of things that must be considered, but reduces the complexity of each of them.

In a similar manner, we use the term control plane when considering how systems are controlled to facilitate desired service consumption patterns. In a control-plane diagram, interactions between Core, Support and application systems are illustrated, but communication between application systems is left out. The reverse holds for data-plane diagrams. In this document we do not separate the two: while we focus on the control plane, the data plane is included for clarity. In Figure 12, message exchange 1 is exclusively concerned with the control plane: A obtains the control-plane details needed to consume *b*. Exchange 2a also belongs to the control plane, as it is between a Core system and an application system. A pure data-plane version would only include the request and response between A and B, without any reference to tokens.

An important aspect of the control plane is that no application data exchange occurs there. The Core systems should not have to consider the requirements of the data transmissions in an Arrowhead local cloud. Cached orchestration and locally verifiable tokens can reduce control-plane traffic while valid credentials, authorisation decisions and encrypted channels remain enforceable. They do not justify shutting down AA dependencies that a provider still needs for identity, permission, token or revocation checks. An unavailable required check fails closed. Control/data-plane separation permits different performance and scaling without removing the v5.0 security baseline.

Support systems can be placed in the same way. The orchestration-support systems, **PlantDescription Engine**, **Choreographer**, **CertificateAuthority**, **Onboarding**, **Configuration**, **SysMonitor** and **Gatekeeper** belong to the control plane: they decide or prepare who talks to whom, with which credentials and settings, and observe the result. **EventHandler**, **DataManager** and **TimeManager** provide application-level services and belong to the data plane. Translation bridges and **GateTunnel** sessions are set up in the control plane but carry data-plane traffic; their capacity must therefore be planned with the data plane. Above the control plane, the **PlantDescriptionEngine** and its engineering UI form a management layer that states how the control plane shall configure the local cloud (Section 4.1).

When designing a local cloud, it is generally useful to split the design into a control and a data plane. The first is concerned with which Core and Support systems, as well as other complementary systems, are used and under what conditions. The second is concerned with how the local cloud directly addresses its business objectives.

7 Interoperability with the Java Reference Implementation

Interoperability is assessed at an explicit service-contract boundary, independently of implementation language. Three different claims must be distinguished: *core-slot substitution* within one local cloud, *federated service consumption* between independently trusted clouds, and *mixed-language providers* registered in the same local cloud. Passing one boundary does not prove complete replacement of every Java API, persistent state format or operational behaviour.

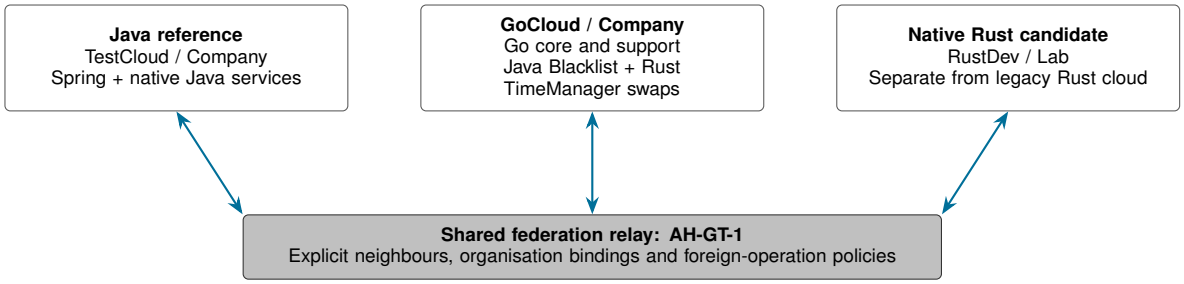


Figure 20: Implementation and trust boundaries. Federated clouds keep their own identities and state. GoCloud additionally demonstrates Java and Rust support providers under its own local-cloud identity and trust configuration.

7.1 Evidence and Current Deployment Scope

Five-core substitution (19–20 October). The Java-cloud interchangeability rerun placed Go and Rust implementations in the same five service slots as Java: ServiceRegistry, Authentication, ConsumerAuthorization, DynamicServiceOrchestration and SimpleStoreServiceOrchestration. The three transitions Go to Rust, Rust to Java and Java to Go passed the recorded core behavioural checks and the same ten Java support-client checks. DNS names, ports, cloud CA and system certificates were retained; each implementation kept separate persistent state. Corrections included the narrowly scoped identity bootstrap lookup, unrestricted bootstrap-discovery metadata, certificate-bound registration/discovery and Java-compatible identity service paths [22]. This dated sequence is evidence of the tested boundary, not an assertion about the currently selected image or database migration.

Native Rust federation and monitoring (19 October). The native `arrowhead-rust-candidate` deployment, identified as `RustDev|Lab`, is distinct from the older `arrowhead-v52-test-cloud-rust` deployment. Its native producer/historian path and AMQP–SysMonitor–MQTT–AIGym pipeline have functional evidence. AH-GT-1 global discovery, negotiation and encrypted HTTP forwarding were exercised with Java and Go peers, including historian read-back, operation/path denial and identity-binding checks. Cross-organisation exchange requires an explicit `RustDev=Lab` peer binding, approved neighbour records and scoped foreign-consumer policies; it does not merge cloud CAs, accounts or business data [21].

Plant-managed orchestration (23 October). GoCloud and the native Rust candidate now implement the Java project's *serviceOrchestrationPlantManagement* extension. Each passed 19 live extension checks, including atomic replacement, invalid-request rejection, persistence, priority/fallback behaviour, unavailable scopes and restriction under both inter-cloud flags. Both final snapshots were empty because no operator plant was active. The legacy Rust cloud has no PDE and was not part of this update. The consoles passed read checks; the report does not establish newly enabled console mutation rights [23].

Mixed-language and MQTT translation (24 October). GoCloud now exposes the Java-compatible MQTT API from all five Go core systems and registers `generic_mqtts`. Reversible swaps run the Java Blacklist with its MQTT API and a native Rust TimeManager with an HTTPS-only API against the Go core. The Go TranslationManager and **TestTranslation** exercise `ALLOW_TRANSLATION` with HTTPS/MQTT interface bridging and SenML-to-`simpleJson` data-model conversion. Recorded paths include MQTT consumption of the Go DataManager, HTTPS consumption of the Go DSO MQTT API, direct/bridged calls to Java Blacklist, and MQTT consumption of Rust TimeManager through a bridge. Wrong bridge tokens, usage reporting and abort were checked; the reported regression passed both with the swaps active and after restoration [24].

Interpreting the source-contract audit. The 23 October audit compared 284 registered Java operations with native registrations and reported 256 exact Go matches and 229 Rust matches. Those are historical service/operation/method/path counts, not current equivalence scores: the 24 October MQTT/translation changes and newer runtime registrations were not re-audited here. The audit identified path-parameter conventions, renamed management operations, policy/version metadata and LocalCloudManager contract

differences; DTO fields and error bodies were not measured. These remain acceptance dimensions rather than claims closed by the passing workload tests [25].

7.2 Contract and Security Requirements

A replacement must preserve the service definition and version, operation names, DTO fields and validation, interface-template properties, policy labels, discovery metadata and externally advertised base paths. HTTPS and MQTT are separate bindings: a compatible HTTPS endpoint alone does not imply MQTT compatibility. MQTT requests retain authentication, blacklist, management and authorisation checks, and responses are confined to the authenticated system's response topic. Identity login/logout/change remain HTTPS-only in the tested Go MQTT profile [24].

The broker certificate is also part of the cross-language contract. GoCloud now issues system-profile subjects for `AmqpBroker.GoCloud.Company.arrowhead.eu` and `MQTTBroker.GoCloud.Company.arrowhead.eu`, while retaining DNS/IP SANs. This satisfies Rust's explicit broker-identity pinning as well as Java/Go/Python hostname verification. Rust TimeManager publishes its native monitoring through AMQP with `messaging.amqpIdentity=AmqpBroker`; it remains an HTTPS-only application provider [24].

7.3 Boundaries and Acceptance of Further Replacements

The evidence establishes the recorded paths, not universal Java equivalence. TestTranslation is a test provider with bounded interface/data-model support; it does not demonstrate arbitrary translators or translation chains. The Rust candidate still lacks complete Java DTO/error/side-effect coverage and complete onboarding provisioning/renewal/off-boarding, advanced choreography, remaining QoS/translation/provider-MQTT integrations and full lifecycle qualification. Its segment-exact tunnel path matching is intentionally stricter than reference descendant matching. The 23 October PDE/DSO update supersedes the older Rust plant-synchronisation limitation, but does not close these other gaps [24, 21, 23].

For each further substitution, record the exact service operations and transports, implementation build, cloud identity, state store and policies. Verify positive consumer/provider calls and denial cases, including token/certificate mismatch, revocation on fresh connections, unauthorised operations and paths, broker topic isolation where applicable, persistence/restart behaviour and rollback to the previous implementation. Stop the replaced service before starting its successor to avoid duplicate identities, DNS aliases and ports. Java Blacklist has its own MySQL store and Rust TimeManager its own state volume; existing Go state is retained, not automatically converted. Passing these checks is functional acceptance within the recorded scope, not production or security certification.

For Go, remaining qualification includes QoS selection, complete push/recovery semantics, golden DTO/error fixtures and state migration. The current support compatibility notes still distinguish SysMonitor's MQTT 3.1.1 output from Java MQTT 5 metadata semantics; the new core MQTT API and translation tests do not by themselves close that telemetry binding gap. Conversely, older catalogue entries claiming no Go MQTT, no Go translation, no native monitoring, no federation or no Go CRL enforcement must not override newer code and dated evidence. The Go security source now defines fail-closed configured CRL checks, and native Rust still documents missing MQTT service APIs and remaining QoS/translation pipelines [11, 24, 21].

8 Release Notes

This section describes the differences between the current version and previous versions of the architecture.

8.1 From Arrowhead 4.x to 5.0

- Every Core and Support system owns its data. The shared SQL database of 4.x is abandoned; 5.x is therefore not backwards compatible with 4.6.1 and its predecessors (Section 9.3).
- Systems are identified by names instead of numeric identifiers; service instances by provider, service definition and version.

- Authentication becomes a Core concern with its own system; orchestration is offered in two approaches with a shared interface.
- Devices and systems become first-class registry content, and interfaces are described by interface templates with operations.

8.2 From 5.0 (GSoSD v1.2) to 5.2.2

- **Systems realised.** The abstract Core systems of the 5.0 GSoSD are realised by **ServiceRegistry**, **Authentication**, **ConsumerAuthorization**, **DynamicServiceOrchestration** (advanced) and **SimpleStoreServiceOrchestration** (atomic).
- **Services renamed and extended.** *Orchestration Pull/Push* become the operations `pull`, `subscribe` and `unsubscribe` of one *serviceOrchestration* service; *Orchestration Management* is split into push, lock, history and store management services; discovery is split into device, system and service discovery; *authorizationToken* and the management services of all systems are added; every system offers *general Management*.
- **Naming convention** [14] enforced by name normalisation and validation.
- **Security.** Certificate, identity-token and (development) self-declared authentication; two-level authorisation rules with metadata-based rules; time-limited, usage-limited and self-contained tokens; restricted direct discovery; protection of all management services by a management policy.
- **Interfaces.** HTTP(S) and MQTT(S) interfaces described by interface templates; every Core service is offered over both when MQTT is enabled. AMQP(S) interface templates are added for the *sysMonitor* service.
- **Advanced orchestration features**: exclusivity locks, matchmaking, preferred providers, blacklist filtering, interface and data-model translation, QoS evaluation and inter-cloud orchestration (Core interfaces released; **Gatekeeper**, **GateTunnel** and the relay as project implementations).
- **Support systems** released upstream: **Blacklist**, **TranslationManager**, **DeviceQoSEvaluator**. Thirteen further support systems have been migrated from 4.6 or newly designed as project implementations [16] (Section 8.3).
- **Port numbering** for local clouds and systems of local clouds is specified (Section 9.7).

8.3 Support Systems from 4.6 to 5.2

Table 5 relates the 5.2 Support systems to their Arrowhead 4.6 predecessors. Common to all migrations: own data store instead of the shared 4.x database, system names instead of numeric identifiers, 5.x authentication and authorisation, services with named operations, the naming convention, and liveness reported through *sysMonitor* instead of an *echo* service. URL paths and ports of 4.6 are kept where possible; the migrated systems are nevertheless not wire compatible with 4.6 clients.

4.6 system	5.2 system	Main architectural changes
QoS Monitor	**DeviceQoSEvaluator**^{U}	Evaluation of devices instead of systems; filter and sort for orchestration
Translator	**TranslationManager**^{U}	Bridges via separate interface and data-model translators
–	**Blacklist**^{U}	New
PlantDescriptionEngine	**PlantDescriptionEngine**	Produces managed DSO provider rules; liveness from **SysMonitor**; alarms on missing monitoring; first engineering UI
Choreographer (core)	**Choreographer**	Becomes a Support system; orchestration for executors by push orchestration; clients start sessions
Configuration (core)	**Configuration**	Proxy access requires authorisation
Certificate Authority	**CertificateAuthority**	Server-enforced end-entity certificates; trusted keys with validity
Onboarding controller, DeviceRegistry, SystemRegistry	**Onboarding**	Registries replaced by the **ServiceRegistry**; complete on- and off-boarding procedure (approval, onboarding, device and system certificates, identities); certificate-only enrolment
EventHandler (core)	**EventHandler**	Identities from authentication
DataManager	**DataManager**	–
TimeManager (core)	**TimeManager**	Trusted only after a successful time check; no WebSocket push
Gatekeeper, Gateway	**Gatekeeper**, **GateTunnel**	Mandatory mutual TLS and identity binding; exchange over an organisation-level relay (MQTT, mutual TLS) with name-constrained cross-certified cloud CAs; sessions per negotiated consumer
–	**SysMonitor**, **AIGym**	New (AIGym from a 4.6 client library)

Table 5: Support systems from 4.6 to 5.2. ^{U} = released with Eclipse Arrowhead 5.2.2.

9 Non-Functional Requirements

Setting up and operating an Arrowhead local cloud comes with the caveats shared by all distributed systems. Managing them typically means prioritising certain qualities when designing or choosing the systems for a local cloud. This section outlines the qualities promoted by the Eclipse Arrowhead project as non-functional requirements.

9.1 Accurate Enough Message Caching

Most modern computing devices, including embedded ones, can call local functions in the order of microseconds or faster. In contrast, calling a function on another device by sending a message typically takes milliseconds or longer. The relative expense, in time and capacity, of sending messages is often high enough to make it worthwhile to reduce the number of messages that must be sent for the use case at hand.

One important way to reduce messaging is to let systems cache data they receive. When a system needs some data, it first consults its cache; if it finds the data there, no message needs to be sent. The trade-off is that cache entries become invalid as soon as the original data change. Unless the caching systems are notified, they cannot be confident the data has not changed without requesting it again. If notification is impractical, cache entries can be expired when too old or when other factors suggest they may be invalid.

Caching can thus reduce messaging, but using it requires planning how to avoid systems acting on invalid entries. The Arrowhead Core and Support systems must be designed to make it easy to reason about and manage the caching of the data they hold. In 5.2.2 this is supported by expiry times on registry entries, time-limited identity and access tokens, self-contained tokens that can be verified without contacting their issuer, short-lived caching of authorisation decisions in providers, and push orchestration, which notifies consumers of changes. Among the Support systems, the **DataManager** proxy serves cached values on behalf of producers, the **SysMonitor** keeps a history so that consumers need not query each monitored system, and systems may cache their configuration and the trusted time between fetches.

9.2 Relative System Independence

Systems tend to become more brittle the larger and more complicated they become, and local clouds are no exception: the more and the more complicated the systems of a local cloud, the higher the probability that one

of them degrades or breaks. This can be mitigated by systems with small scopes, so that fewer things can break in each, and by making systems able to work around such eventualities independently. Independence means that each system needs to own enough resources and capabilities, which among other things entails systems owning their own data rather than storing it in shared external databases. Every 5.2.2 Core and Support system owns its data store.

Designing a local cloud effectively means that its resilience to failures must be accounted for, and that its systems are given the capabilities required to work around contingencies.

9.3 Backwards Compatibility

For major releases some backwards incompatibility may occur. From 4.x.y to 5.0.0 the Core and Support systems became responsible for their own data; 4.x.y used a common SQL database, which is not the case for 5.0.0 onwards. 5.x.y Core systems are therefore not backwards compatible with 4.6.1 and its predecessors. The general approach to backwards compatibility follows these guidelines:

- A 5.x+1.y release is based on a new Core system being added or a major update of one of the released Core systems. A 5.x.y+1 release contains bug fixes.
- 4.6.1 is maintained for at least 12 months after the release of 5.0.0.
- 5.1 shall be backwards compatible with 5.0; 5.2 with 5.0 and 5.1; 5.3 with 5.0, 5.1 and 5.2.

Migrated 4.6 Support systems keep their 4.6 URL paths and ports where possible but are not wire compatible with 4.6 clients, since identities, payloads and management interfaces follow 5.x.

9.4 Authentication Mechanisms

Eclipse Arrowhead has the ambition to support a number of authentication mechanisms. For 5.x the major approach is X.509 certificates following the Arrowhead certificate profile. Release 5.2.2 adds identity tokens issued by the **Authentication** system, and the combination of both. Self-declared identities exist for development only. With the **Onboarding** system, the certificates and identities themselves are issued through a controlled procedure that starts from a manufacturer bootstrap certificate approved by an operator (Section 4.2). Further mechanisms such as OpenID Connect and OAuth will be investigated and decided upon in due course.

9.5 Naming Convention for Local Clouds, Microsystems, Microservices, . . .

A naming convention has been agreed by the Roadmap WG [14]; the earlier proposal by Cristina Paniagua and Jerker Delsing from 2019 was considered too complex. Release 5.2.2 applies it as follows:

Identifier	Form	Example
Microsystem (system name)	PascalCase, ASCII letters and digits	`DynamicServiceOrchestration`
Microservice (service definition)	camelCase, ASCII letters and digits	`serviceOrchestration`
Service operation	lowercase kebab-case	`modify-priorities`
Interface template	lowercase snake_case	`generic_mqtts`

Table 6: Naming convention as applied in 5.2.2.

Deployment identifiers – software packages, URL paths, container and host names, database names – keep the forms required by their platforms and are not Arrowhead system or service names.

9.6 Active vs. Passive Systems

The Arrowhead Core Framework supports both active and passive systems.

Active systems act on their own behalf and can at any time change consumed or produced services. They interact with the Core Framework to obtain the information they need, typically by pull orchestration.

Passive systems normally wait for some external stimulus before acting. They perform their duty and then return to a waiting state. Push orchestration lets the Core Framework deliver the stimulus.

Consequences for the Core Framework: it must support systems ranging from very passive to fully active, and it must be resilient to systems that make their own decisions in terms of, for example, authorisation and orchestration.

9.7 Port Numbering

Systems find each other through the **ServiceRegistry**, so only its port must be known for bootstrapping. Well-known default ports nevertheless matter for co-location, firewalls and running several local clouds side by side. The following rules apply; the complete specification and allocation table are given in the Core GSoSDD [1].

- Every Arrowhead framework system type has one canonical port, unique across the framework and used for all its HTTP(S) services; allocated ports are stable, and ports of retired systems remain reserved.
- A local cloud uses the port block 8000–8999, divided into ranges: tunnel control pool (8000–8099), inter-cloud session endpoints (8200–8299), life-cycle and engineering systems (8430s), Core and trust systems (8440s; **ServiceRegistry** 8443), support systems (8450–8469), monitoring and QoS (8470s), health endpoints (8480s), operator tools (8490s), application systems (8500–8799) and messaging (8883 MQTT over TLS). The AMQP broker for *sysMonitor* uses the well-known AMQP-over-TLS port 5671. The inter-cloud relay is organisation infrastructure outside the blocks of the local clouds.
- Local clouds sharing hosts or addresses in a system of local clouds are given a cloud index $k \in \{0, \ldots, 23\}$ and use port $=$ canonical $+ 1000 \cdot k$, which keeps all ports below the ephemeral range.

10 Unresolved Issues

The major discussion points #1–10 of the roadmap reached acceptable conclusions in December 2023, which were presented at a workshop in Luleå, Sweden, on 24 January 2024. The realisation of the architecture in release 5.2.2 has raised the following issues, which are proposed for discussion in the Roadmap WG:

1. **Orchestration backend convergence.** PDE now uses the local DSO managed-rule extension, so plant engineering no longer requires SSO beside DSO. Upstream release remains to be decided. Go and native Rust implemented and tested the extension on 23 October; wider contract equivalence remains scoped by Section 7.
2. **Management-role separation.** The Java console now requires operator login and uses host/origin checks and a constrained Docker proxy. Separating engineering from operations remains an architectural question; console mutation rights are not inferred from the native clouds' read-only PDE verification.
3. **Monitoring service.** A standard liveness and telemetry service for all microsystems is not yet part of the released architecture. The *sysMonitor* contract (Section 3.7) has been specified and implemented in all 5.2 systems as a project proposal and should be considered for release.
4. **Translation coverage.** GoCloud now has the TestTranslation test system, exercising HTTPS/MQTT bridges and SenML/simpleJson conversion, including Java and Rust providers. Arbitrary translator chains and universal interface/data-model compatibility remain unproven [24].
5. **User authentication** and authentication methods beyond X.509 certificates and identity tokens (e.g. OpenID Connect, OAuth) remain open.
6. **Inter-cloud orchestration.** The project Gatekeeper/GateTunnel implementations now use AH-GT-1 encrypted exchange with explicit peer and operation bindings. Java/Go/Rust functional paths are documented in Section 7; upstream release, exhaustive contract/security review and organisation-level relay credential lifecycle remain open.
7. **Monitoring transport.** *sysMonitor* uses AMQP while *sysMonitorData* and other MQTT traffic use the MQTT broker, so a local cloud runs two brokers. Whether *sysMonitorData* should also move to AMQP is open.
8. **Authorisation by intermediaries.** The authorisation model lets a provider verify its consumers, but an intermediary (for example an event handler) cannot verify on behalf of publishers and subscribers.
9. **Security-baseline conformance.** Full AA and encrypted information exchange are the default from v5.0 (Section 2.2). Any component sample or test configuration that disables TLS, trusts declared names or bypasses authorisation is a development exception and must not be presented as the architectural default. Release and deployment checks must verify conformance before exposing services.
10. **Onboarding.** The announcement-based onboarding of the Support GSoSD [4] is realised with pre-distributed addresses; the role of announcements should be decided. Device approval relies on the bootstrap certificate fingerprint; hardware attestation is not yet used. The Java workspace Onboarding implementation now rejects reserved framework names for devices and hosted systems; the configured reservation set and cross-implementation coverage require continued review.
11. **Certificate life cycle.** The deployed Java CA publishes CRLs and services and brokers reject revoked cloud certificates on new handshakes. Established connections and identity sessions need separate termination. Automated renewal, verified key activation, root rollover and the organisation-level relay-server revocation lifecycle require further qualification [20, 21].
12. **Availability of stateful Support systems.** Several Support systems are designed for a single active instance; replication requires shared stores.
13. **Release scope of Support systems.** The twelve active project-implemented Support systems should be reviewed for inclusion in an upstream release.

12 References

[1] J. Delsing, “Generic System of Systems Design Description: Arrowhead Core Systems 5.2.2 – White Box, v1.1,” 2026, roadmap/5.2/Arrowhead SoS architecture/WhiteBox_5.2.2.

[2] ——, “Generic System of Systems Design Description: Arrowhead Support Systems 5.2 – White Box, v0.12,” 2026, roadmap/5.2/Arrowhead SoS architecture/WhiteBox_5.2.2.

[3] Eclipse Arrowhead Roadmap WG, “Generic System-of-Systems Description (GSoSD): The Arrowhead Core Systems 5.0, v1.2,” 2025, roadmap/5.2/Arrowhead SoS architecture/BlackBox_5.2.2/GSoSD Arrowhead Core Systems 5.0 v1.2.docx (approved by the Roadmap WG 2025-07-09).

[4] ——, “Generic System-of-Systems Description (GSoSD): The Arrowhead Support Systems 5.0 (draft 0.1),” 2023, roadmap/5.2/Arrowhead SoS architecture/OLD/GSoSD Arrowhead Support Systems 5.0.docx.

[5] EISLAB Arrowhead workspace, “Primary source policy and implementation inventories,” 2026, 30 October. Eclipse/PRIMARY-SOURCES.md; primary Java, Go and Rust README files. Local source designation, not an upstream release or automatic deployment.

[6] ——, “Independent Java support distribution verification,” 2026, eclipse-Arrowhead-java: JAVA-SUPPORT-FOLDERS.md and ALL-JAVA-SUPPORT-VERIFICATION-2026-10-25.md. Fifteen standalone projects; isolated builds/startup, not new full live-cloud acceptance.

[7] ——, “Go reorganisation verification,” 2026, eclipse-Arrowhead-go/verification/REPORT.md, 28 October. Twenty-two modules verified; optional native-registry integration failure retained. Later LocalCloudConfigurator is outside that audit.

[8] ——, “Rust workspace layout and compatibility boundaries,” 2026, eclipse-Arrowhead-rust/README.md; core and support COMPATIBILITY.md; LocalCloudManager/VERIFICATION.md. Five core entrypoints, fifteen shared-crate support binaries and a separate manager crate.

[9] ——, “Java LocalCloudConfigurator and graphical local-cloud editor,” 2026, eclipse-Arrowhead-java/ah5-LocalCloudConfigurator-java: README.md, documentation and verification/2026-10-30/REPORT.md. Baseline live workflow is verification-only.

[10] ——, “Integrated microsystem database assets,” 2026, java LocalCloudConfigurator verification/2026-10-30/database-assets/REPORT.md; catalog and runtime-isolation evidence. Sixteen private host-owned database services; 20 Java tests, ten focused manager/editor tests and ten browser groups.

[11] ——, “Native implementation compatibility and security boundaries,” 2026, sources inspected 24 October: ah5-core-go/docs/COMPATIBILITY.md and SECURITY.md; ah5-support-go/COMPATIBILITY.md; ah5-core-rust/COMPATIBILITY.md; ah5-support-rust/COMPATIBILITY.md. Later dated deployment evidence takes precedence over superseded gap lists.

[12] ——, “Java graphical engineering and SoS operations tools,” 2026, primary Java LocalCloudManager: SOS-CONFIGURATION.md, GRAPHICAL-ORCHESTRATION.md and LOCAL-CLOUD-CONFIGURATION.md; ah5-SoSConfigurator-java/README.md. Source inspection 30 October.

[13] ——, “Roadmap source and runtime inspection,” 2026, 30 October. roadmap/5.2/IMPLEMENTATION-UPDATE-2026-10-30.md and verification evidence. Java cloud running-service inventory: 44 containers; GraphDBManager remains a deployment extension outside the three supplied primary source inventories.

[14] Eclipse Arrowhead Roadmap WG, “Naming convention for local clouds, microsystems and microservices (Naming_convention_2),” 2025, roadmap/5.2/Arrowhead SoS architecture/Naming_convention.

[15] Eclipse Arrowhead, “ah5-blacklist-java-spring, ah5-translation-manager-java-spring, ah5-device-qos-evaluator-java-spring, release v5.2.2,” https://github.com/eclipse-arrowhead, 2026.

[16] EISLAB, Luleå University of Technology, “ah5-support-java-spring: Arrowhead 5.2.2 support system migrations from v4.6,” 2026, workspace implementations, not upstream releases.

[17] A. Bicaku, S. Maksuti, C. Hegedűs, M. Tauber, J. Delsing, and J. Eliasson, “Interacting with the Arrowhead Local Cloud: On-boarding Procedure,” in *Proc. 2018 IEEE Industrial Cyber-Physical Systems (ICPS)*, 2018.

[18] EISLAB, Luleå University of Technology, “SysMonitor AMQP: Interface Design Description SysMonitor-AMQP-IDD-5.2.2,” 2026, ah5-support-java-spring/SysMonitor/Document/LaTeX.

[19] ——, “Arrowhead inter-cloud relay,” 2026, arrowhead-relay (Eclipse Mosquitto, mutual TLS, organisation PKI).

[20] EISLAB Arrowhead workspace, “Java TestCloud deployment and orchestration snapshot,” 2026, 24 October. Arrowhead-test-cloud-java: compose.yaml, README.md and service-policy scripts; live Docker project arrowhead-v52-test-cloud, 22 microsystems and 41 containers.

[21] ——, “Native Rust federation and functional acceptance,” 2026, 19 October. Arrowhead-test-cloud-rust/candidate-native/FUNCTIONAL-FEDERATION.md and REPLACEMENT-STATUS.md; later PDE results supersede the earlier plant-synchronisation limitation.

[22] ——, “Interchangeability rerun, 19–20 October 2026,” 2026, arrowhead-test-cloud-java/interchangeability/RESULTS-2026-10-19.md. Five core slots, three interleaved implementation transitions and unchanged Java support clients.

[23] ——, “Go and native Rust PDE/DSO deployment,” 2026, 23 October. security-evaluation/pde-dso-go-rust-2026-10-23/README.md. Nineteen live extension checks passed in each cloud; console verification was read-only.

[24] ——, “GoCloud: MQTT API, translation bridges, Java and Rust support systems,” 2026, 24 October. Arrowhead-test-cloud-go/MQTT-2026-10-24.md; companion TRANSLATION-2026-10-24.md. Dated engineering test evidence.

[25] ——, “Arrowhead 5.2.2 Java, Go and Rust interchangeability,” 2026, 23 October. Local source-contract comparison, retained in the Interchangability analysis directory. Historical operation counts predate the 24 October MQTT/translation updates; DTO and error-body parity were not measured.

[26] Eclipse Arrowhead, “ah5-core-java-spring, release v5.2.2,” https://github.com/eclipse-arrowhead/ah5-core-java-spring, 2026.

[27] AITIA International, “Arrowhead Framework 5 – Java Spring implementation documentation (SysD, SD, IDD),” https://aitia-iiot.github.io/ah5-docs-java-spring/home/welcome/, 2026.

[28] EISLAB, Luleå University of Technology, “ah5-serviceregistry-go: Arrowhead 5.2.2 ServiceRegistry in Go,” 2026, workspace implementation, not an upstream release.

[29] J. Delsing, Ed., *IoT Automation: Arrowhead Framework*. Boca Raton, FL: CRC Press, 2017.

[30] Eclipse Arrowhead, “Arrowhead SysML profile and library,” https://github.com/eclipse-arrowhead/profile-library-sysml.

13 Revision History

13.1 Amendments

No.	Date	Rev.	Subject of Amendments	Author
1	2023-01-12	0.1	Initial draft.	Per Olofsson
2	2023-02-03	0.2	Refined and extended Sections 1, 2 and 3. Removed unneeded sections. Cleaned up the other.	Emanuel Palm
3	2023-02-07	0.3	Update figures. Update Sections 1, 2. Write Sections 4, 6 and 7.	Emanuel Palm
4	2023-02-24	0.4	Added to Section 4.	Per Olofsson
5	2023-09-24	0.5	Updated according to Issue #3, 5, 7, 8.	Jerker Delsing
6	2023-09-25	0.6	Updates according to Issue #2.	Raymond Bocsi
7	2023-09-27	0.7	Updated according to Roadmap discussions.	Per Olofsson
8	2024-03-07	1.0	Updated the introduction reflecting the ambition to provide a SOA/microservice architecture infrastructure. Declaring this v1.0 of the Eclipse Arrowhead GSoSD 5.0 document.	Jerker Delsing
9	2024-07-04	1.1	Update related to priority between application microsystem authorisation vs authentication system.	Jerker Delsing
10	2025-07-09	1.2	Added naming convention with reference to agreed document.	Jerker Delsing
11	2026-10-13	1.3	Converted to the Arrowhead LaTeX template and updated to release 5.2.2 from the Core and Support implementations and their documentation: realised systems and services, two orchestration approaches, Authentication system, new authenticated-access use case, updated use cases and figures, Support systems overview, release notes, port numbering, system-service matrix (Appendix A), unresolved issues.	Jerker Delsing
12	2026-10-13	1.4	Extended to the Support systems based on the 5.2.2 Support implementations and their documentation: new Section 3 (Support microsystems), six Support use cases, Support systems in the control/data planes, release notes from 4.6 to 5.2, caching, unresolved issues; title changed to Core and Support Systems; supersedes the draft GSoSD Arrowhead Support Systems 5.0.	Jerker Delsing
13	2026-10-15	1.5	Management layer and security mechanisms (new Section 4): the operational PlantDescriptionEngine with its first engineering UI; Onboarding as admission control with the complete on- and off-boarding procedure (Bicaku et al. 2018); updated PlantDescriptionEngine and Onboarding descriptions, use cases, release notes and unresolved issues.	Jerker Delsing
14	2026-10-15	1.6	sysMonitor over AMQP (broker, addressing, access rules; *sysMonitorData* stays on MQTT); inter-cloud transport over the organisation-level relay with name-constrained cross-certified cloud CAs (new Section 3.8.3, inter-cloud use case, security mechanisms, release notes, ports, unresolved issues).	Jerker Delsing
15	2026-10-23	1.7	PDE backend migrated from SSO to DSO managed snapshots; updated dependencies, use cases, security constraints and migration requirements.	Codex (draft)

16	2026-10-24	1.8	Added deployed Java architecture and service paths; documented Java/Go/Rust substitution, federation, PDE/DSO and mixed-language MQTT translation evidence; corrected superseded deployment observations.	Codex (draft)
17	2026-10-24	1.9	Applied the SysML v1.6 Papyrus library colour convention to all system diagrams, sequence participants and deployment graphics; added a shared palette and legend.	Codex (draft)
18	2026-10-24	2.0	Rewrote bootstrapping and corresponding security descriptions: full AA and encrypted information exchange are the default from v5.0; unsecured test modes are explicit exceptions.	Codex (draft)
19	2026-10-30	2.1	Updated primary Java/Go/Rust packaging and evidence limits, four-core configuration default, graphical engineering tools and integrated private database ownership.	Codex (draft)

13.2 Quality Assurance

No.	Date	Version	Approved by
1	2025-07-09	1.2	Roadmap WG
2		1.6	

A System-Service Matrix

A.1 Core Systems

Table 8 shows, for each service definition, which Core system produces (P) or consumes (C) it. Consumption marked C* is conditional on configuration (e.g. identity-token authentication, authorisation-based management policy, blacklist filtering, or the corresponding orchestration feature being enabled).

Service definition	Service-Registry	Authenti-cation	Consumer-Authorization	Dynamic-Orch.	SimpleStore-Orch.
serviceDiscovery	P	C	C	C	C
systemDiscovery	P	C	C	C	C
deviceDiscovery	P				
serviceRegistryManagement	P				
identity	C*	P	C*	C*	C*
identityManagement		P			
authorization	C*	C*	P	C*	C*
authorizationToken			P		
authorizationManagement			P	C*	
authorizationTokenManagement			P	C*	
serviceOrchestration				P	P
serviceOrchestrationPushManagement				P	P
serviceOrchestrationLockManagement				P	
serviceOrchestrationHistoryManagement				P	P
serviceOrchestrationSimpleStore Management					P
generalManagement	P	P	P	P	P
blacklistDiscovery (**Blacklist**)	C*		C*	C*	C*
blacklistManagement (**Blacklist**)				C*	
translationBridgeManagement (**Translation Manager**)				C*	
qualityEvaluation (**DeviceQoSEvaluator**)				C*	

Table 8: System-service matrix of the Core systems (P = produces, C = consumes).

A.2 Support Systems

Table 9 lists the service definitions produced by the Support systems (Section 3) and their main consumers. Every Support system also consumes the registration, identity and authorisation services of the Core and produces *generalManagement*.

Producer	Service definitions	Main consumers
Blacklist^U	*blacklistDiscovery*, *blacklistManagement*	all systems (request filter), **DynamicService Orchestration**, **TranslationManager**
TranslationManager^U	*translationBridge*, *translationBridge Management*, *translationReport*	**DynamicServiceOrchestration**; interface translators (*translationReport*)
DeviceQoSEvaluator^U	*qualityEvaluation*, *deviceQualityData Management*	**DynamicServiceOrchestration**; operators
PlantDescriptionEngine	*plantDescriptionManagement*, *plant DescriptionMonitor*	operators, engineering tools
Choreographer	*choreographerSession*, *choreographer Executor*, *choreographerManagement*	application clients, executors, operators
Configuration	*configuration*, *configurationProxy*, *configurationManagement*	all systems (own configuration), operators

Producer	Service definitions	Main consumers
CertificateAuthority	*certificateSigning*, *certificateValidation*, *certificateAuthorityManagement*	**Onboarding**, operators, any system (validation)
Onboarding	*onboarding*, *onboardingManagement*	new devices and their systems; operators (approval, off-boarding)
EventHandler	*eventPublish*, *eventSubscription*, *event HandlerManagement*	application systems
DataManager	*dataProxy*, *dataHistorian*	application systems
TimeManager	*time*	any system
SysMonitor	*sysMonitorHistory*, *sysMonitorData*, *sys MonitorManagement*	**PlantDescriptionEngine**, **AIGym**, analytics, operators
AIGym	*aiGym*	experimenters
Gatekeeper	*globalServiceDiscovery*, *interCloud Negotiation*, *gatekeeperManagement*	orchestration (inter-cloud), neighbour **Gatekeeper**s
GateTunnel	*gateTunnelConnection*, *gateTunnelKey*, *gate TunnelManagement*	**Gatekeeper**, operators

Table 9: Services produced by the Support systems. ^{U} = released with Eclipse Arrowhead 5.2.2.

A.3 Implementations

The matrix of the 5.0 GSoSD also maps systems to their available implementations. Table 10 lists those known at release 5.2.2.

System	Implementation	Status
ServiceRegistry	Java/Spring, `ah5-core-java-spring` [26]	upstream release 5.2.2
	Go, `ah5-serviceregistry-go` [28]	project implementation
Authentication, **Consumer Authorization**, **DynamicService Orchestration**, **SimpleStore ServiceOrchestration**	Java/Spring, `ah5-core-java-spring` [26]	upstream release 5.2.2
Blacklist, **TranslationManager**, **DeviceQoSEvaluator**	Java/Spring, one repository each [15]	upstream release 5.2.2
Remaining Support systems	Independent Java/Spring or native Java projects in the primary distribution [6]	project implementations

Table 10: Known implementations of the 5.2.2 systems. SDs and IDDs are published with each implementation [27].